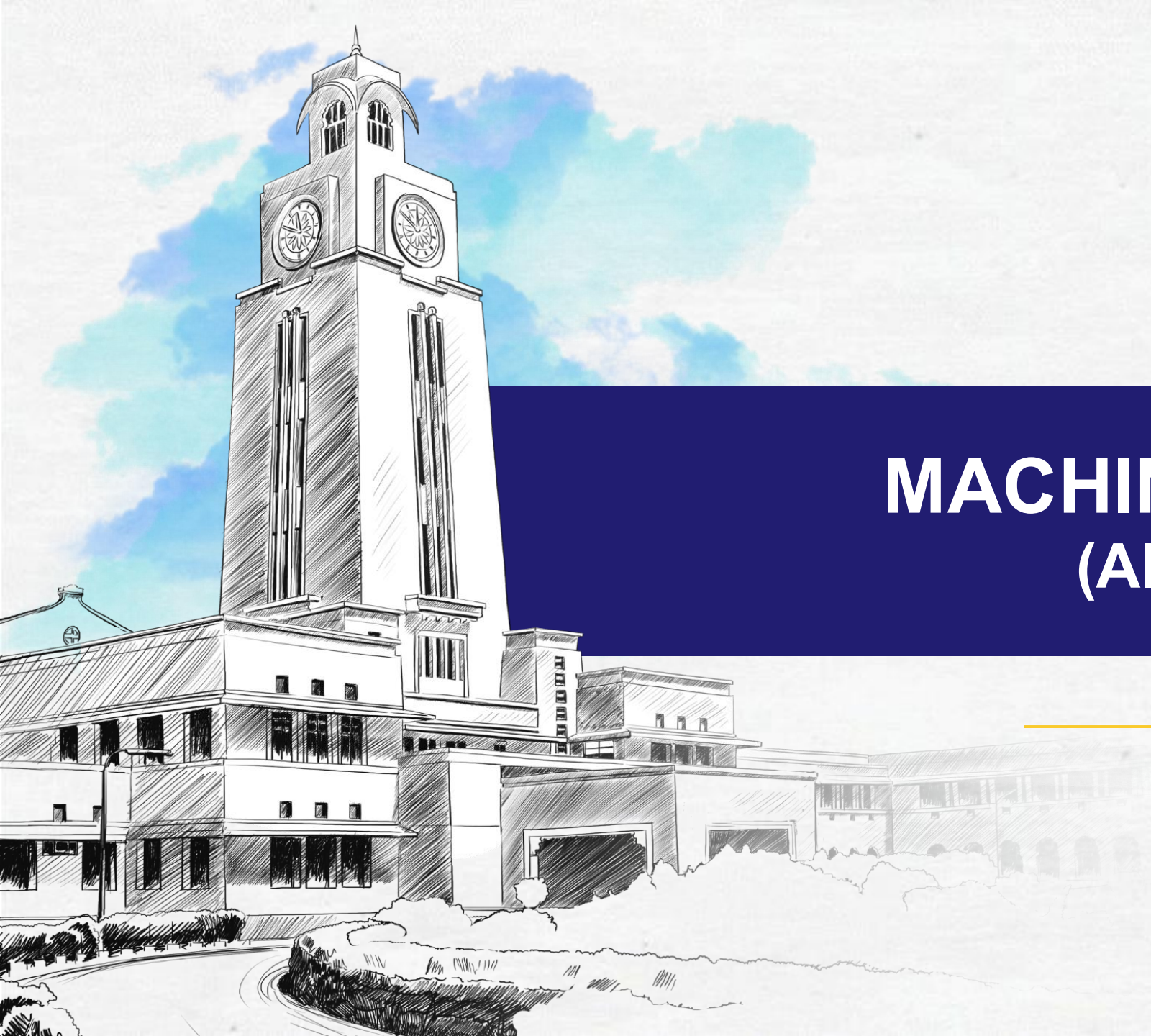




BITS Pilani
Pilani | Dubai | Goa | Hyderabad

MACHINE LEARNING (AIML ZG565)

1



Session 7 & 8
(6th & 12th September, 2026)

Course Plan

M1	Introduction
M2	Machine learning Workflow
M3	Linear Models for Regression
M4	Linear Models for Classification
M5	Decision Tree
M6	Instance Based Learning
M7	Support Vector Machine
M8	Bayesian Learning
M9	Ensemble Learning
M10	Unsupervised Learning
M11	Machine Learning Model Evaluation/Comparison

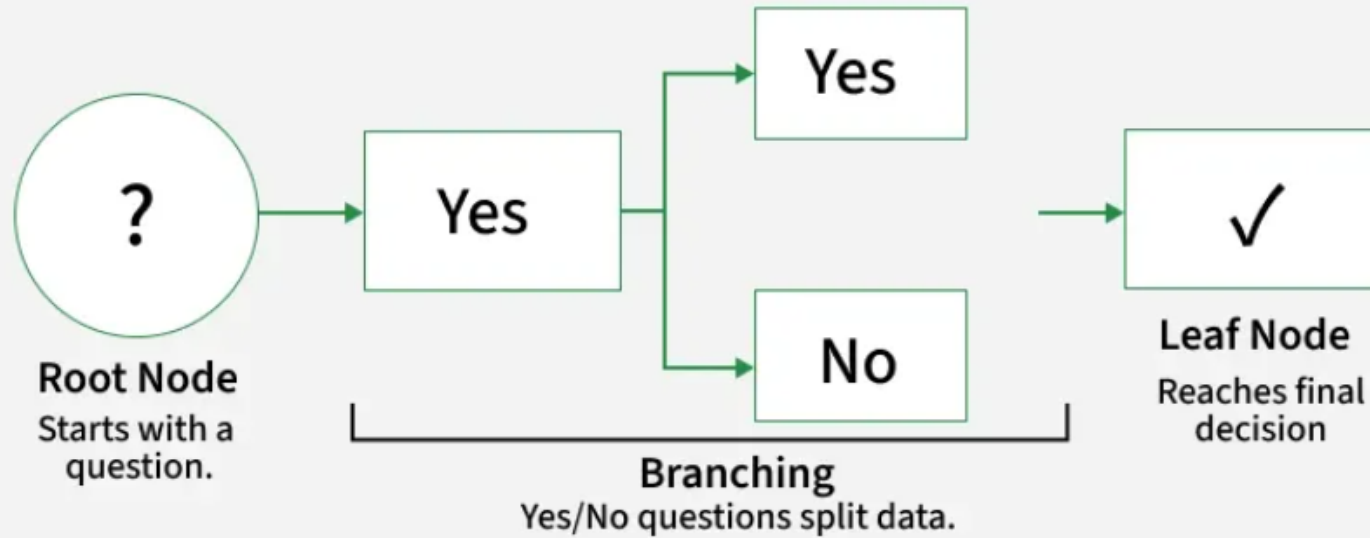


Decision Tree Classifier

Decision Tree

- The decision tree is a hierarchical tree-based algorithm that is used to classify or predict outcomes based on a set of rules. It works by splitting the data into subsets based on the values of the input features.
- The Decision tree algorithm works by selecting the best feature to split the data at each node.
- It has a tree-like structure that starts with one main question called the **root node** which represents the entire dataset. From there, the tree branches out into different possibilities based on features in the data.

How Decision Trees Work?

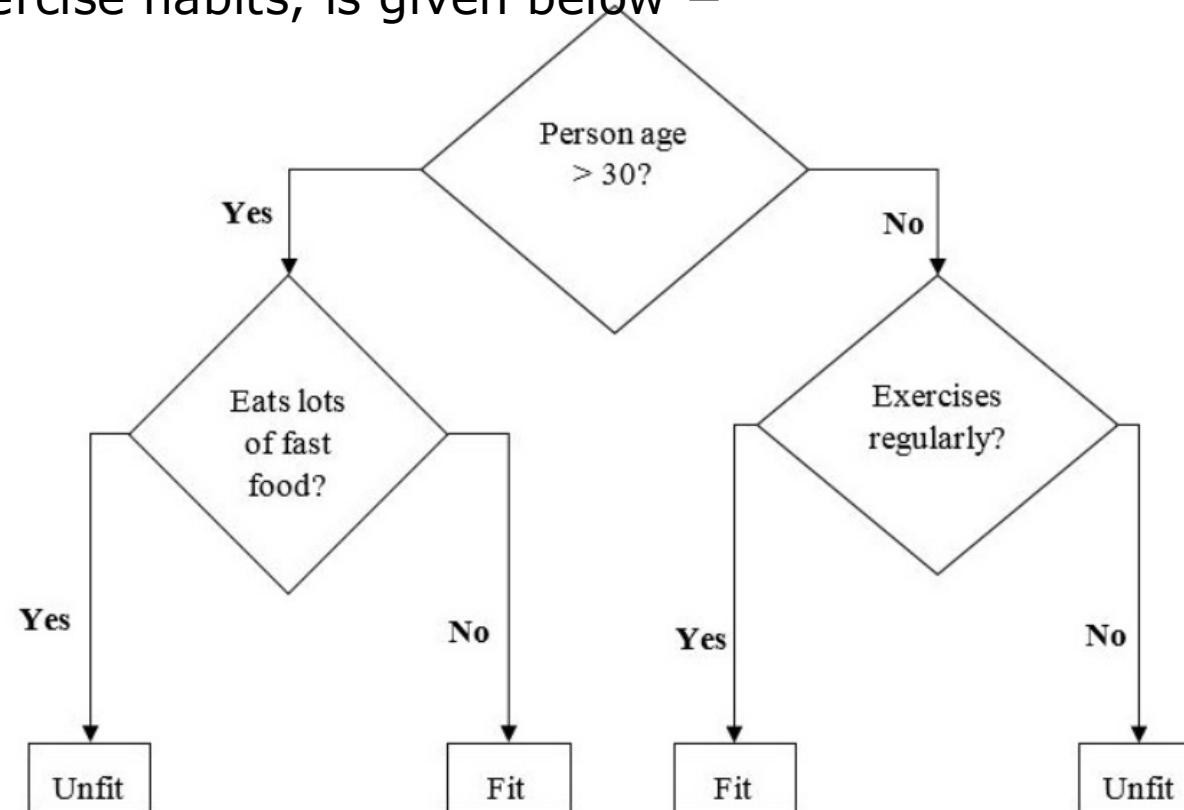


Building a **Decision tree**

- **Root Node:** Has no incoming edges, Zero or more outgoing edges
- **Internal Nodes:** Each has exactly one incoming edge, two or more outgoing edges
- **Leaf (Terminal) Nodes:** Each has exactly one incoming edge, No outgoing edges
 - Each leaf node assigned a class label
 - Each non-leaf node contains a test condition on one of the attributes
- The resulting tree is a set of decision rules that can be used to make predictions or classify new data.

Example:

A binary tree for predicting whether a person is fit or unfit providing various information like age, eating habits and exercise habits, is given below –



Types of Decision Tree Algorithm

There are two main types of Decision Tree algorithm –

- **Classification Tree** – A classification tree is used to classify data into different classes or categories. It works by splitting the data into subsets based on the values of the input features and assigning each subset to a different class.
- **Regression Tree** – A regression tree is used to predict numerical values or continuous variables. It works by splitting the data into subsets based on the values of the input features and assigning each subset a numerical value.

Decision Tree Construction - Hunt's Algorithm

A decision tree is grown **recursively** by partitioning training records into **purier subsets**.

Step 1:

- If all records at a node belong to the same class → make the node a **leaf** labeled with that class.

Step 2:

- If records belong to more than one class → choose the **best attribute test condition** to split them.
- Create child nodes for each outcome, and distribute records.
- Recursively apply Hunt's algorithm to each child.

Decision Tree

Example: Deciding Whether to Play Tennis

- We want to decide whether it is a good day to play tennis.
- One approach is to pose a series of questions about the weather conditions.

First Question: What is the Outlook?

- If **Overcast** → Yes (play tennis)
- If **Sunny** → Ask Humidity
 - High → No (do not play)
 - Normal → Yes (play)
- If **Rain** → Ask Wind
 - Strong → No (do not play)
 - Weak → Yes (play)

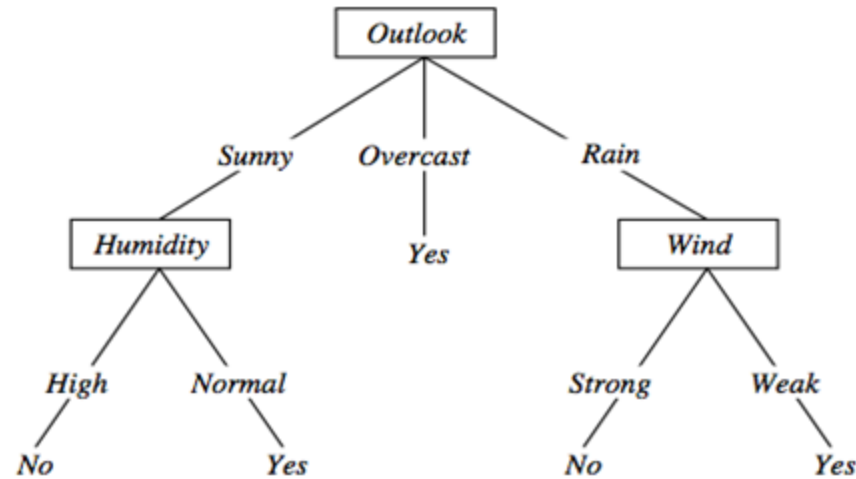
Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Key Idea:

By asking this sequence of questions, the decision tree helps us classify any new day into **Play Tennis = Yes or No**.

- Ask a series of **questions** about the **attributes** of the test record.
- Each answer leads to a follow-up question **until we reach a decision**.
- These questions and answers are organized into a **decision tree**, a hierarchical structure with nodes and directed edges.

Decision Theory: Interpretation



Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

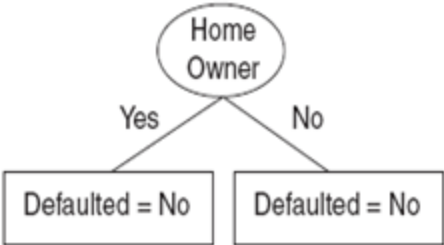
- **Root Node:** Has no incoming edges, Zero or more outgoing edges
- **Internal Nodes:** Each has exactly one incoming edge, Two or more outgoing edges
- **Leaf (Terminal) Nodes:** Each has exactly one incoming edge, No outgoing edges
 - ❖ Each leaf node assigned a class label
 - ❖ Each non-leaf node contains a test condition on one of the attributes

Decision Tree Construction: Example

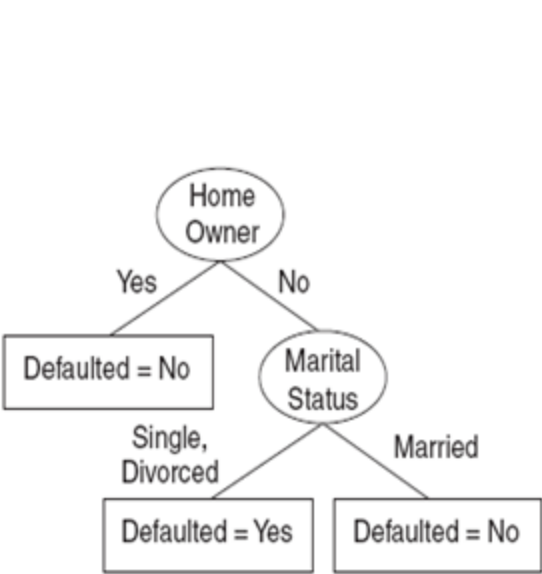
	binary	categorical	continuous	class
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Defaulted = No

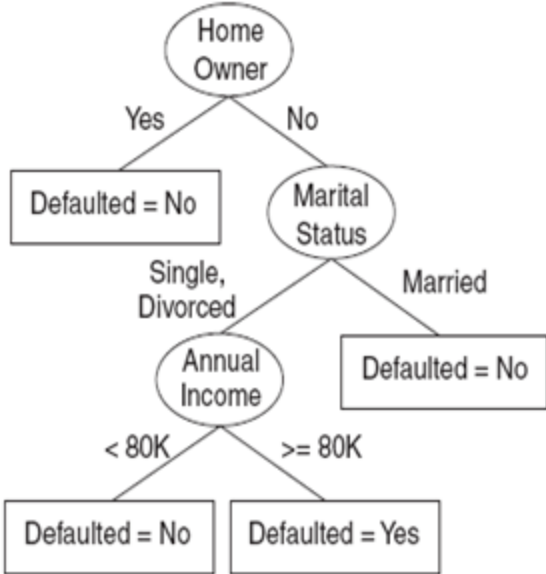
(a)



(b)



(c)

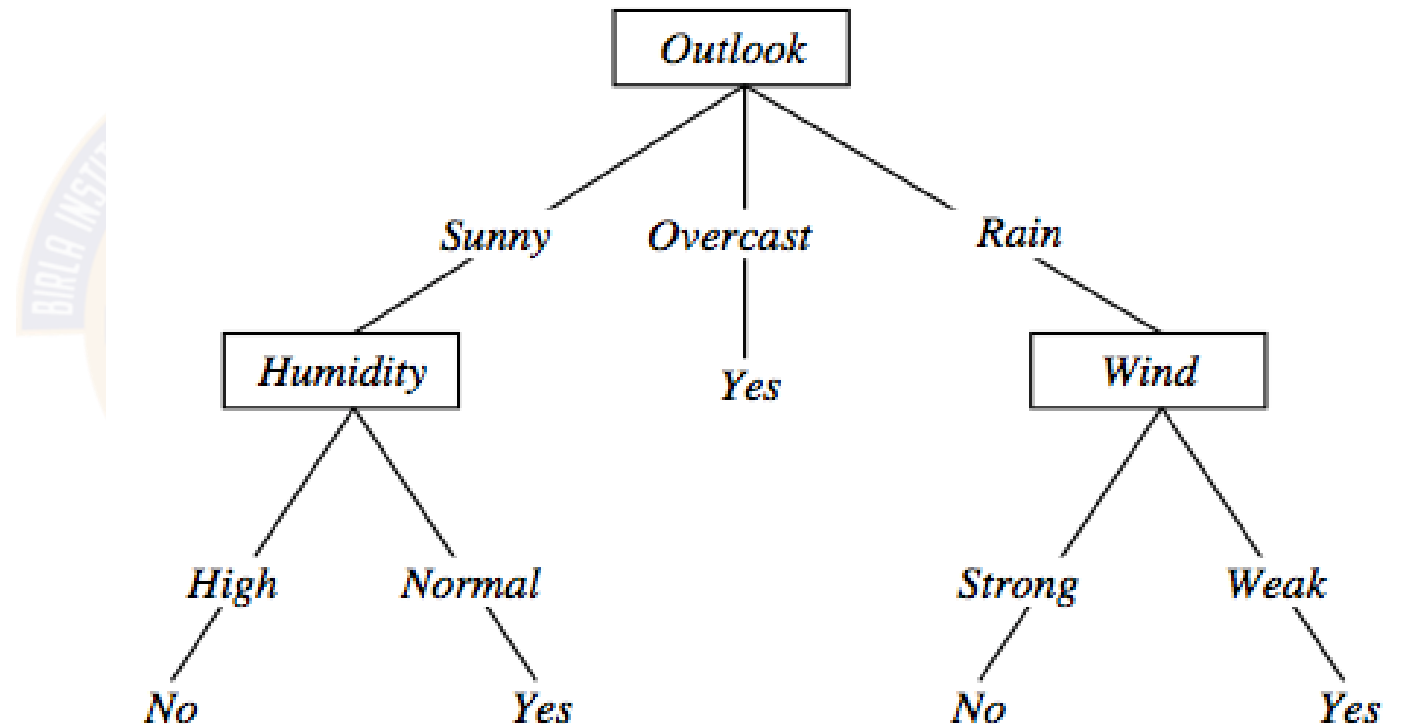


(d)

Decision Trees – Representation & Expressiveness

Decision trees represent a disjunction of conjunctions on constraints on the value of attributes:

$(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \vee (\text{Outlook} = \text{Overcast}) \vee (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak})$



$\langle \text{Outlook}=\text{Sunny}, \text{Humidity}=\text{High}, \text{Wind}=\text{Strong} \rangle$ **No**

Design Issues in Decision Tree Induction

How to Split the Records?

- At each recursive step, choose an **attribute test condition**.
 - Define test conditions for different **attribute types** (categorical, continuous).

When to Stop Splitting?

- Stopping criteria are needed to prevent infinite growth.
 - All records at a node belong to the same class.
 - All records have identical attribute values.
 - Other criteria (e.g., maximum depth, minimum records per node) to stop earlier and avoid overfitting.

Methods for Expressing Attribute Test Conditions for different attribute type

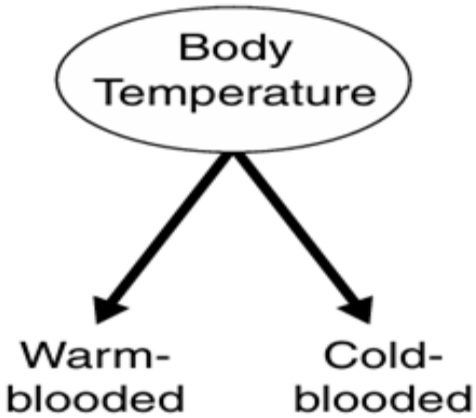


Figure 4.8. Test condition for binary attributes.

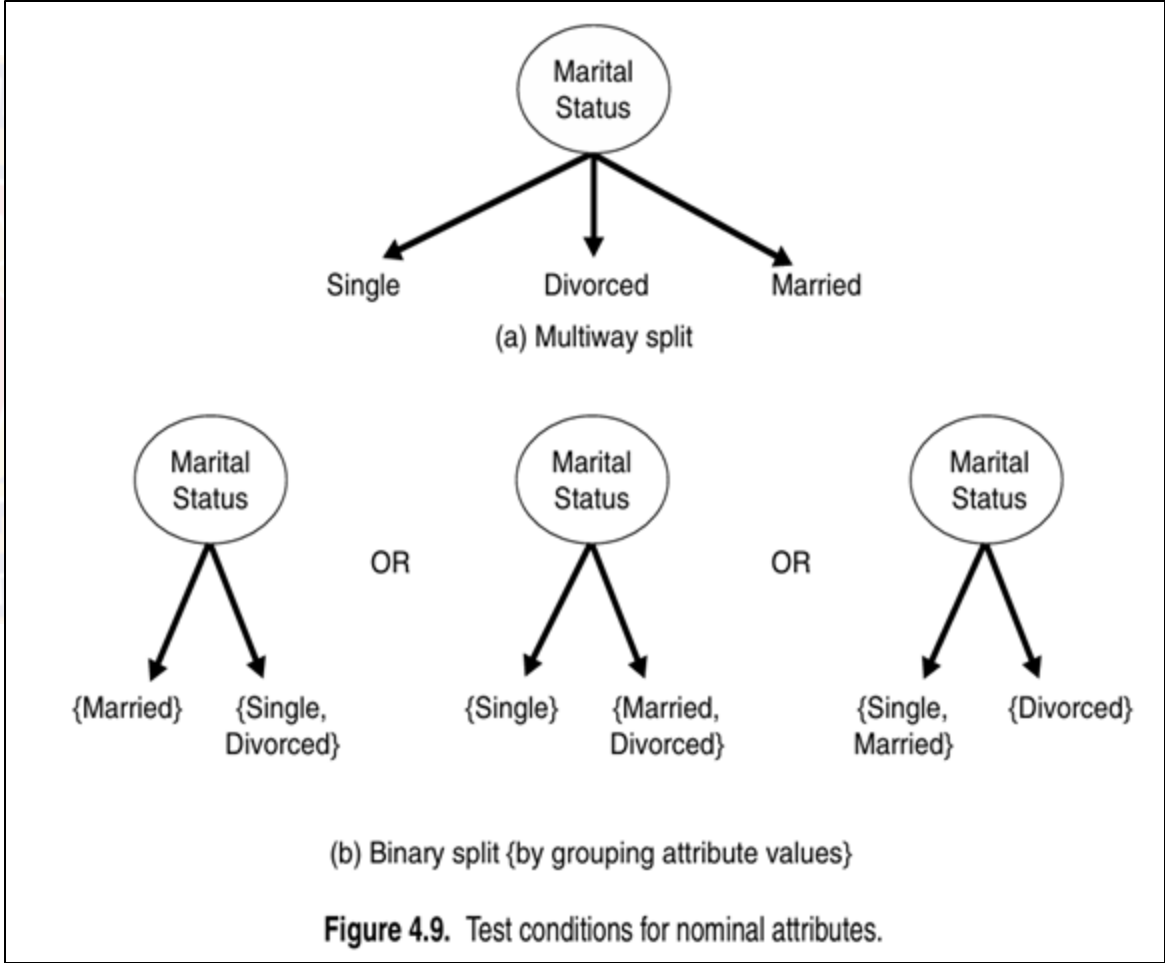
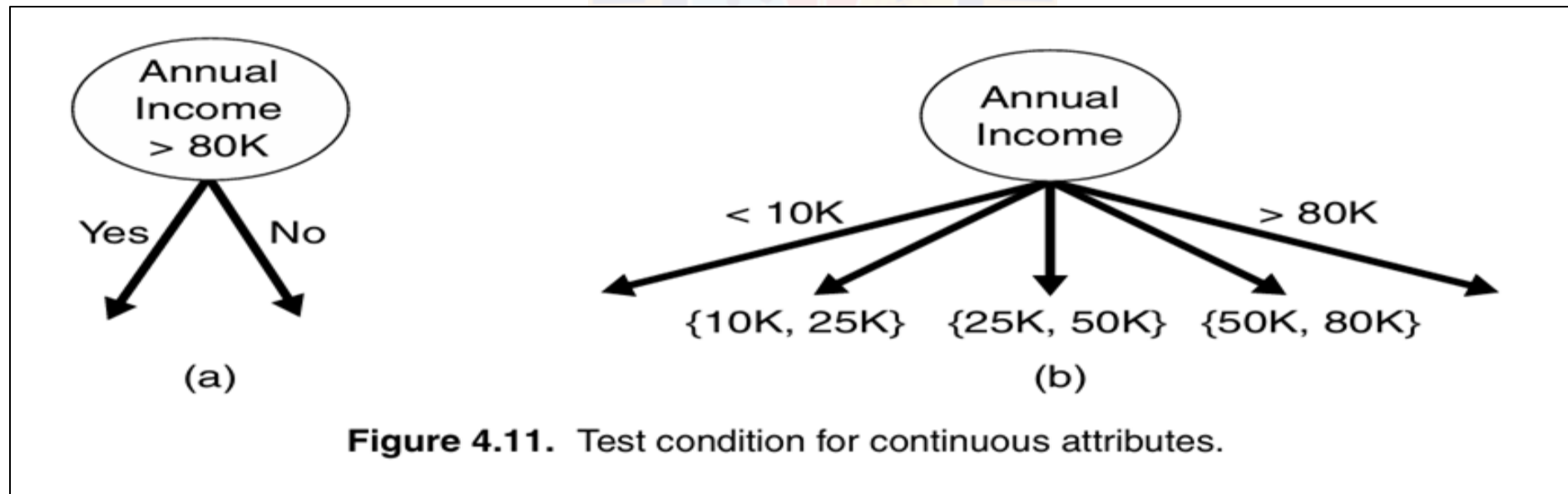
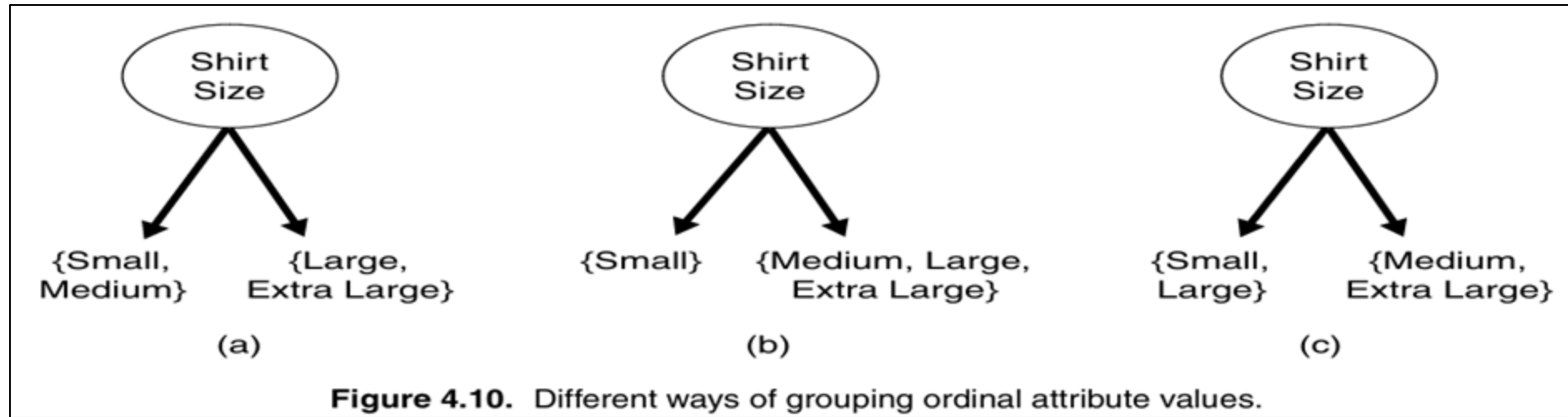


Figure 4.9. Test conditions for nominal attributes.

Methods for Expressing Attribute Test Conditions



Measures for Selecting the Best Split

- **Impurity** measures how **mixed** the class labels are in a node.

A node is:

- **Pure** if all records belong to the same class $\rightarrow$ impurity = 0.
- **Impure** if records are evenly split between classes $\rightarrow$ impurity is highest.

Before Splitting: 10 records of class 0, 10 records of class 1

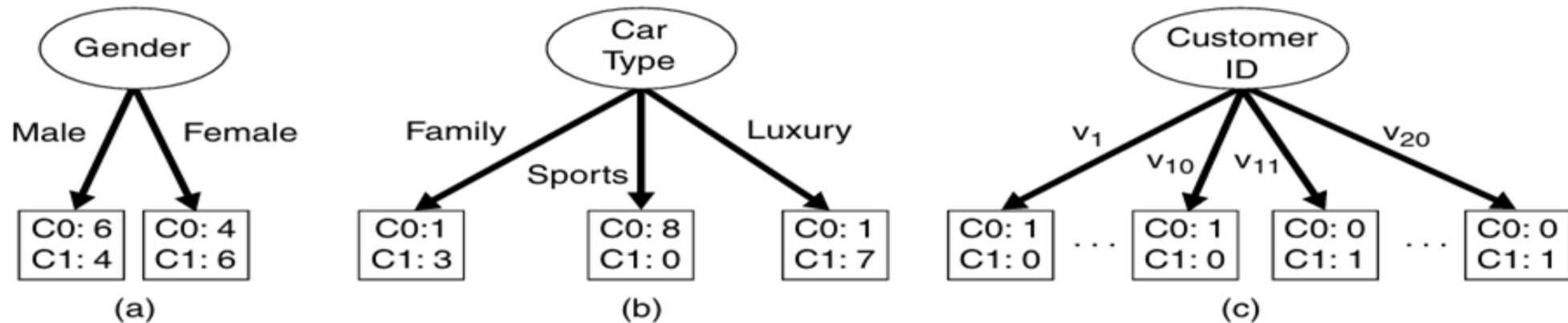


Figure 4.12. Multiway versus binary splits.

Which test condition is the best?

How to Measure Impurity?

- Entropy
- Gini Index
- Classification Error



Entropy

- **Entropy** is a measure of **impurity (uncertainty)** in a dataset (node).
- It quantifies the **amount of disorder** or **randomness** in the class distribution of records at a node.

$$Entropy(t) = - \sum_{i=1}^C p(i|t) \log_2 p(i|t)$$

Where:

- t = a node in the decision tree
- C = total number of classes
- i = class index (1, 2, ..., C)
- $p(i|t)$ = fraction (probability) of records at node t that belong to class i
- $\log_2$ = base-2 logarithm (measured in **bits**)

- **Entropy = 0** → node is **pure** (all records same class).
- **Entropy is maximum** when classes are evenly distributed

Entropy in binary Classification

- Entropy measures the *impurity* of a collection of examples. It depends from the distribution of the random variable p .
 - S is a collection of training examples
 - p_+ the proportion of positive examples in S
 - p_- the proportion of negative examples in S

$$\text{Entropy}(S) \equiv -p_+ \log_2 p_+ - p_- \log_2 p_-$$

Node N_1	Count
Class=0	0
Class=1	6

$$\text{Entropy} = -(0/6) \log_2(0/6) - (6/6) \log_2(6/6) = 0$$

Node N_2	Count
Class=0	1
Class=1	5

$$\text{Entropy} = -(1/6) \log_2(1/6) - (5/6) \log_2(5/6) = 0.650$$

Node N_3	Count
Class=0	3
Class=1	3

$$\text{Entropy} = -(3/6) \log_2(3/6) - (3/6) \log_2(3/6) = 1$$

Note: the log of a number < 1 is negative, $0 \leq p \leq 1, 0 \leq \text{entropy} \leq 1$

Information gain as entropy reduction

- *Information gain* is the *expected* reduction in entropy caused by partitioning the examples on an attribute.
- The higher the information gain the more effective the attribute in classifying training data.
- Expected reduction in entropy knowing A

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

$S \rightarrow$ the set of training examples

$A \rightarrow$ the attribute being tested

$Values(A) \rightarrow$ possible values of attribute A

S_v subset of S for which A has value v

Maximizing the gain = minimizing the weighted average impurity measure of children nodes

Example

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Example: Information gain

Parent node $S = [9^+, 5^-]$

$$\begin{aligned} Entropy(S) &= -\left(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14}\right) \\ &= -(0.6429 \cdot \log_2 0.6429 + 0.3571 \cdot \log_2 0.3571) \\ &= 0.9403 (\approx 0.94) \end{aligned}$$

Child nodes after splitting on Wind

$S_{\text{Weak}} = [6^+, 2^-]$:

$$Entropy(S_{\text{Weak}}) = -\left(\frac{6}{8} \log_2 \frac{6}{8} + \frac{2}{8} \log_2 \frac{2}{8}\right) = 0.8113 (\approx 0.811)$$

$S_{\text{Strong}} = [3^+, 3^-]$:

$$Entropy(S_{\text{Strong}}) = -\left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6}\right) = 1.0000 (= 1.00)$$

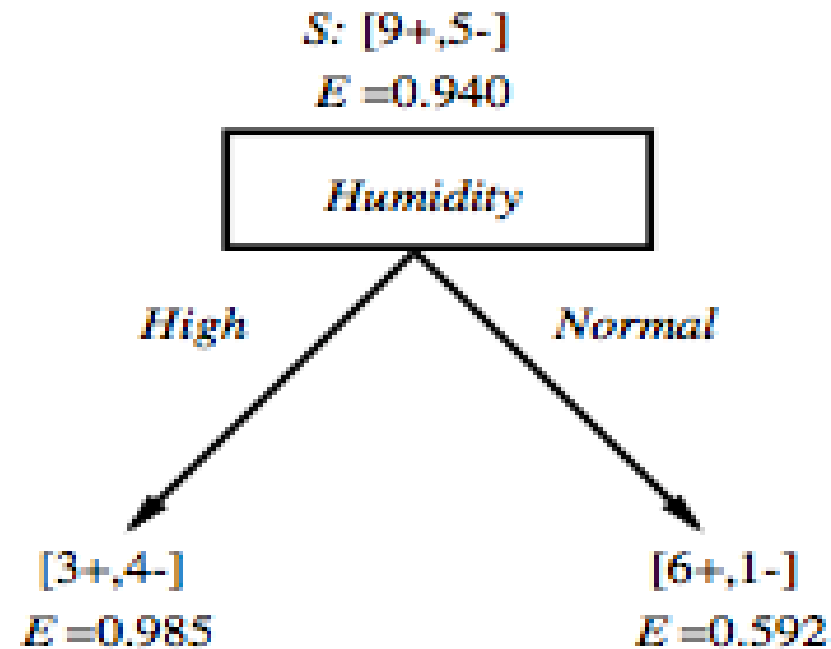
Information Gain for Wind

$$\begin{aligned} Gain(S, \text{Wind}) &= Entropy(S) - \frac{8}{14} Entropy(S_{\text{Weak}}) - \frac{6}{14} Entropy(S_{\text{Strong}}) \\ &= 0.9403 - \frac{8}{14} \cdot 0.8113 - \frac{6}{14} \cdot 1.00 \\ &= 0.0481 (\approx \mathbf{0.048}) \end{aligned}$$

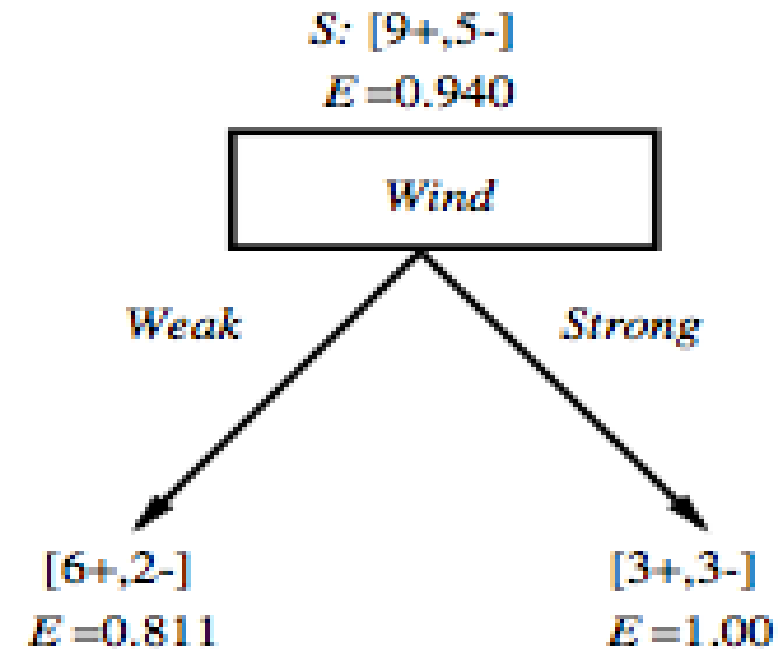
Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Which attribute is the best classifier?

Which attribute is the best classifier?



$$\begin{aligned} \text{Gain}(S, \text{Humidity}) &= .940 - (7/14) \cdot .985 - (7/14) \cdot .592 \\ &= .151 \end{aligned}$$



$$\begin{aligned} \text{Gain}(S, \text{Wind}) &= .940 - (8/14) \cdot .811 - (6/14) \cdot 1.0 \\ &= .048 \end{aligned}$$

First step: which attribute to test at the root?

Entropy of the whole dataset S

We have 14 instances.

- PlayTennis = Yes $\rightarrow$ 9
- PlayTennis = No $\rightarrow$ 5

$$Entropy(S) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right)$$

$$Entropy(S) = -\frac{9}{14} \cdot 0.485 - \frac{5}{14} \cdot 0.737$$

$$Entropy(S) = 0.940$$

Gain(S, Outlook)

Outlook has 3 values: Sunny, Overcast, Rain.

- Sunny (5 instances) $\rightarrow$ 2 Yes, 3 No

$$Entropy(Sunny) = -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971$$

- Overcast (4 instances) $\rightarrow$ 4 Yes, 0 No

$$Entropy(Overcast) = 0$$

- Rain (5 instances) $\rightarrow$ 3 Yes, 2 No

$$Entropy(Rain) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

Weighted entropy:

$$Entropy_{Outlook} = \frac{5}{14} (0.971) + \frac{4}{14} (0) + \frac{5}{14} (0.971) = 0.693$$

$$Gain(S, Outlook) = 0.940 - 0.693 = 0.247$$

First step: which attribute to test at the root?

Gain(S, Humidity)

Humidity has 2 values: High, Normal.

- High (7 instances) → 3 Yes, 4 No

$$Entropy(High) = -\frac{3}{7} \log_2 \frac{3}{7} - \frac{4}{7} \log_2 \frac{4}{7} = 0.985$$

- Normal (7 instances) → 6 Yes, 1 No

$$Entropy(Normal) = -\frac{6}{7} \log_2 \frac{6}{7} - \frac{1}{7} \log_2 \frac{1}{7} = 0.592$$

Weighted entropy:

$$Entropy_{Humidity} = \frac{7}{14} (0.985) + \frac{7}{14} (0.592) = 0.789$$

$$Gain(S, Humidity) = 0.940 - 0.789 = 0.151$$

Gain(S, Wind)

Wind has 2 values: Weak, Strong.

- Weak (8 instances) → 6 Yes, 2 No

$$Entropy(Weak) = -\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8} = 0.811$$

- Strong (6 instances) → 3 Yes, 3 No

$$Entropy(Strong) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1.0$$

Weighted entropy:

$$Entropy_{Wind} = \frac{8}{14} (0.811) + \frac{6}{14} (1.0) = 0.892$$

$$Gain(S, Wind) = 0.940 - 0.892 = 0.048$$

First step: which attribute to test at the root?

Gain(S, Temperature)

Temperature has 3 values: Hot, Mild, Cool.

- Hot (4 instances) → 2 Yes, 2 No

$$Entropy(Hot) = 1.0$$

- Mild (6 instances) → 4 Yes, 2 No

$$Entropy(Mild) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = 0.918$$

- Cool (4 instances) → 3 Yes, 1 No

$$Entropy(Cool) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.811$$

Weighted entropy:

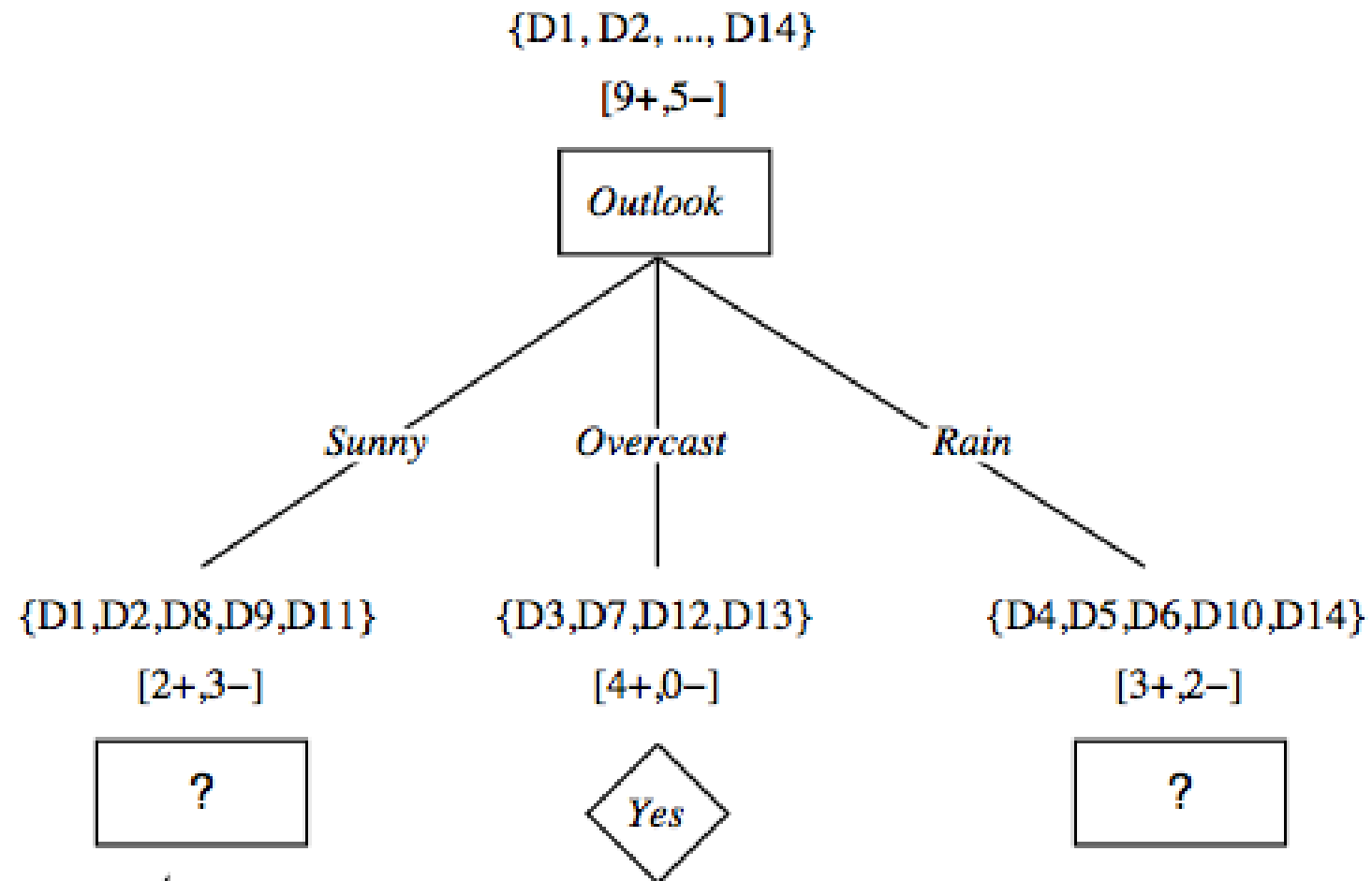
$$Entropy_{Temp} = \frac{4}{14} (1.0) + \frac{6}{14} (0.918) + \frac{4}{14} (0.811) = 0.911$$

$$Gain(S, Temperature) = 0.940 - 0.911 = 0.029$$

First step: which attribute to test at the root?

- Which attribute should be tested at the root?
 - $Gain(S, Outlook) = 0.247$
 - $Gain(S, Humidity) = 0.151$
 - $Gain(S, Wind) = 0.048$
 - $Gain(S, Temperature) = 0.029$
- **Outlook** provides the best prediction for the target
- Lets grow the tree:
 - add to the tree a successor for each possible value of *Outlook*
 - partition the training samples according to the value of *Outlook*

After first step



Second step

- Working on *Outlook=Sunny* node:

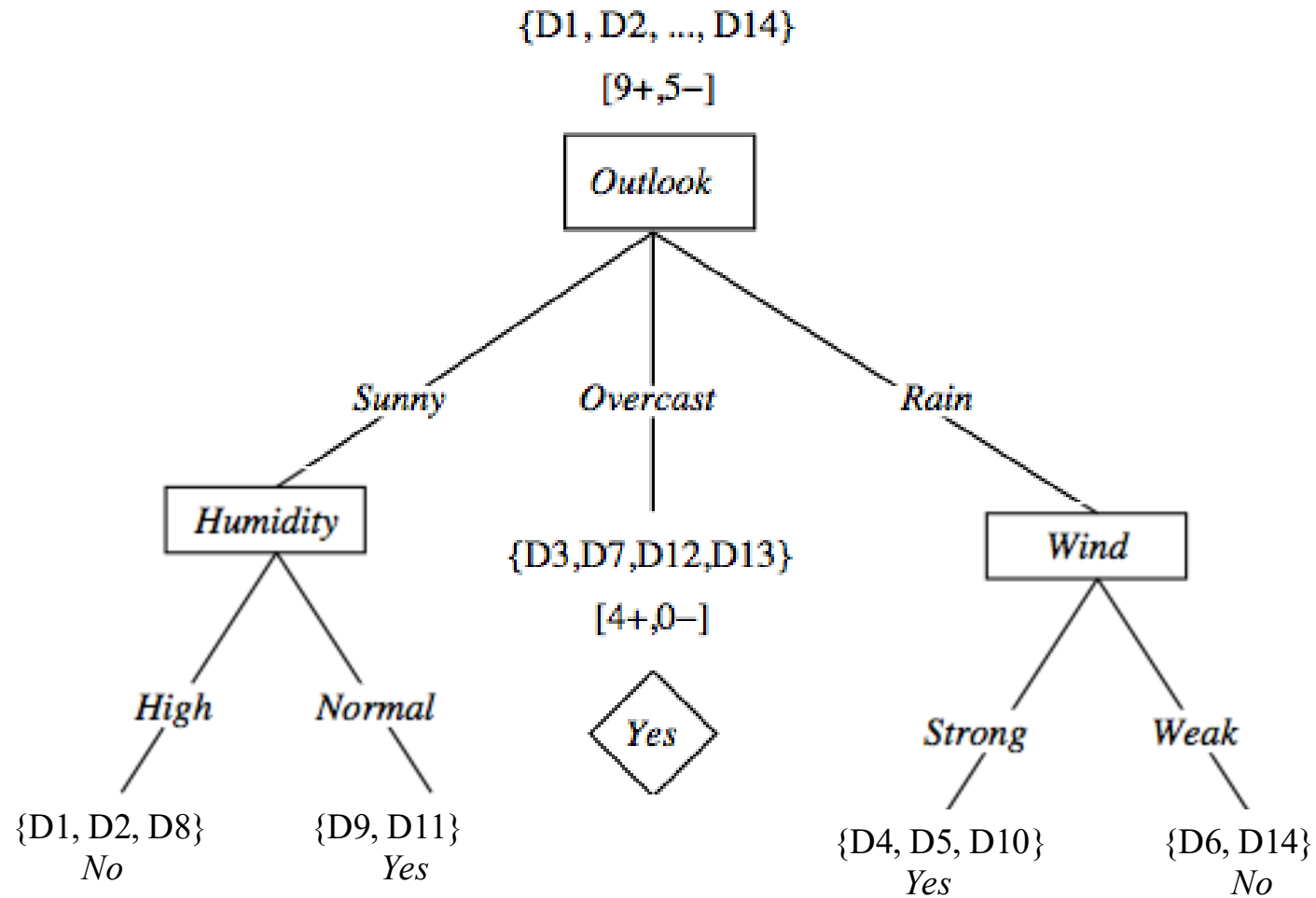
$$\text{Gain}(S_{\text{Sunny}}, \text{Humidity}) = 0.971 - 3/5 \times 0.0 - 2/5 \times 0.0 = \mathbf{0.971}$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Wind}) = 0.971 - 2/5 \times 1.0 - 3/5 \times 0.918 = 0.020$$

$$\text{Gain}(S_{\text{Sunny}}, \text{Temp.}) = 0.971 - 2/5 \times 0.0 - 2/5 \times 1.0 - 1/5 \times 0.0 = 0.571$$

- **Humidity** provides the best prediction for the target
- Lets grow the tree:
 - add to the tree a successor for each possible value of *Humidity*
 - partition the training samples according to the value of *Humidity*

Second and third steps



ID3: algorithm -Summary

ID3(X , T , $Attrs$) X : training examples:
 T : target attribute (e.g. *PlayTennis*),

$Attrs$: other attributes, initially all attributes

Create Root node

If all X 's are +, *return* Root with class +

If all X 's are -, *return* Root with class -

If $Attrs$ is empty *return* Root with class most common value of T in X

else

$A \leftarrow$ best attribute; decision attribute for Root $\leftarrow A$

For each possible value v_i of A :

 - add a new branch below Root, for test $A = v_i$

 - $X_i \leftarrow$ subset of X with $A = v_i$

 - *If* X_i is empty *then* add a new leaf with class the most common value of T in X

else add the subtree generated by ID3(X_i , T ,
 $Attrs - \{A\}$)

return Root

Decision trees

➤ **Advantages:**

- Inexpensive to construct
- Extremely fast at classifying unknown records
- Easy to interpret for small-sized trees
- Can easily handle redundant or irrelevant attributes (unless the attributes are interacting)

➤ **Disadvantages:**

- Space of possible decision trees is exponentially large.
- Greedy approaches are often unable to find the best tree.
- Does not take into account interactions between attributes
- Each decision boundary involves only a single attribute

Decision Trees – Problem Type 1

Entropy of the Decision

Feature EnjoySport:

Target Attribute (Decision Class)

- The column **EnjoySport?** is the class label (Yes/No).
- We have **4 examples**:
 - Yes $\rightarrow$ 3
 - No $\rightarrow$ 1

$$= \text{Entropy}_{\text{DecisionClass}}(\mathbf{S}) = \sum -p(c) \log_2 p(c)$$

$$= E(3,1) = - (3/4) \log_2 (3/4) - (1/4) \log_2 (1/4) = 0.4421 + 0.5 = 0.94$$

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Note:

- Class balancing is recommended before building
- Entropy is always between 0 to log base 2 k, where k is the number of classes you have.

Decision Trees – Problem Type 1

Entropy of the Decision
Feature EnjoySport:

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

$$\begin{aligned} &= \text{Entropy}_{\text{DecisionClass}}(\mathbf{S}) = \sum p(c) \log_2 p(c) \\ &= E(3,1) = - (3/4) \log_2 (3/4) - (1/4) \log_2 (1/4) = 0.4421 + 0.5 = 0.94 \end{aligned}$$

Entropy of the attribute Sky:

$$\begin{aligned} &= p(\text{Sunny}) \cdot E(3,0) + p(\text{Rainy}) \cdot E(0,1) \\ &= (3/4) [- (3/3) \log_2 (3/3) - (0/3) \log_2 (0/3)] + (1/4) [- (0/1) \log_2 (0/1) - (1/1) \log_2 (1/1)] \\ &= 0 \end{aligned}$$

Sky	Enjoy	Not Enjoy
Sunny	3	0
Rainy	0	1

Entropy of the attribute AirTemp:

$$\begin{aligned} &= p(\text{Warm}) \cdot E(3,0) + p(\text{Cold}) \cdot E(0,1) \\ &= (3/4) [- (3/3) \log_2 (3/3) - (0/3) \log_2 (0/3)] + (1/4) [- (0/1) \log_2 (0/1) - (1/1) \log_2 (1/1)] \\ &= 0 \end{aligned}$$

Air Temp	Enjoy	Not Enjoy
Warm	3	0
Cold	0	1

Decision Trees – Problem Type 1

Entropy of the Decision

Feature EnjoySport:

$$\begin{aligned} &= \text{Entropy}_{\text{DecisionClass}}(\mathbf{S}) = -\sum p(c) \log_2 p(c) \\ &= E(3,1) = - (3/4) \log_2 (3/4) - (1/4) \log_2 (1/4) = 0.94 \end{aligned}$$

Entropy of the attribute Humidity :

$$\begin{aligned} &= p(\text{Normal}) \cdot E(1,0) + p(\text{High}) \cdot E(2,1) \\ &= (1/4) [- (1/1) \log_2 (1/1) - (0/1) \log_2 (0/1)] + (3/4) [- (2/3) \log_2 (2/3) - (1/3) \log_2 (1/3)] \\ &= 0 + 3/4 [0.39 + 0.52] = 0.68 \end{aligned}$$

Entropy of the attribute Wind :

$$\begin{aligned} &= p(\text{Strong}) \cdot E(3,1) \\ &= (4/4) [- (3/4) \log_2 (3/4) - (1/4) \log_2 (1/4)] \\ &= 1(0.94) = 0.94 \end{aligned}$$

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Humidity	Enjoy	Not Enjoy
Normal	1	0
High	2	1

Wind	Enjoy	Not Enjoy
Strong	3	1

Decision Trees – Problem Type 1

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Entropy of the Decision

Feature EnjoySport:

$$= \text{Entropy}_{\text{DecisionClass}}(\mathbf{S}) = \sum p(c) \log_2 p(c)$$

$$= E(3,1) = - (3/4) \log_2 (3/4) - (1/4) \log_2 (1/4) = 0.94$$

Entropy of the attribute Forecast :

$$= p(\text{Same}) \cdot E(2,0) + p(\text{Change}) \cdot E(1,1)$$

$$= (2/4) [- (2/2) \log_2 (2/2) - (0/2) \log_2 (0/2)] + (2/4) [- (1/2) \log_2 (1/2) - (1/2) \log_2 (1/2)]$$

$$= 0 + 0.5 = 0.5$$

Forecast	Enjoy	Not Enjoy
Same	2	0
Change	1	1

Decision Trees – Problem Type 1

Entropy of the Decision Feature *EnjoySport*: **0.94**

Entropy of the attribute *Sky*: **0**

Entropy of the attribute *AirTemp*: **0**

Entropy of the attribute *Humidity*: **0.68**

Entropy of the attribute *Wind*: **0.94**

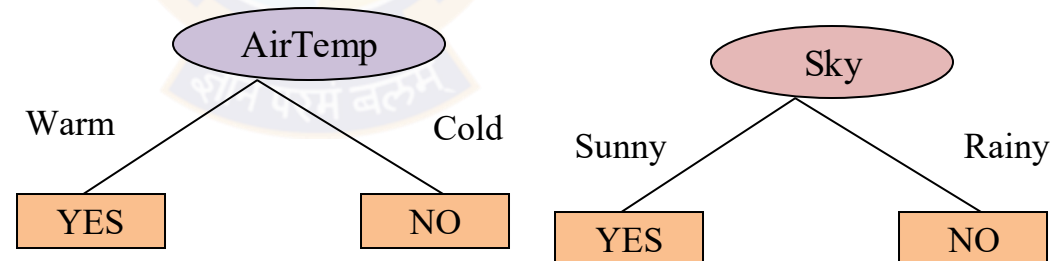
Entropy of the attribute *Forecast*: **0.5**

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport ?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Attribute that generalizes the model faster = That which reduces the randomness if used to decide = That which have the highest info gain = SKY & AIRTEMP

Decision Tree constructed

(both are equally correct decision tree) :



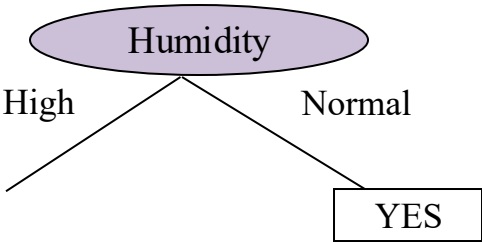
Decision Trees – Problem Type 2

(If there are heterogeneous nodes, continue the decision tree construction)

Sub Problem which has more scope to Model: None

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes

Assumed sample data in the node



Convergence Criteria :

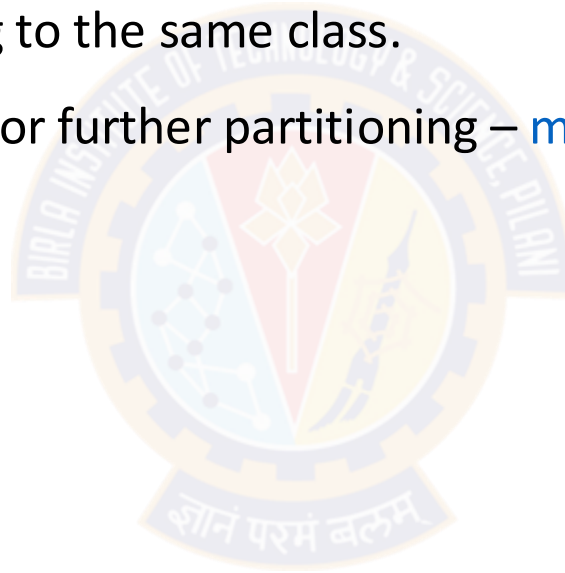
Repeat the step again for the remaining features till:

- All training data are learnt
- No more learning is possible
- Constraints posed by the hyperparameter satisfied
- (More on this will be discussed in handling overfitting)

Sky	AirTemp	Wind	Forecast	EnjoySport?
Sunny	Warm	Strong	Same	Yes
Rainy	Cold	Strong	Change	No
Sunny	Warm	Strong	Change	Yes

Convergence of Algorithm

- Conditions for stopping partitioning.
- All samples for a given node belong to the same class.
- There are no remaining attributes for further partitioning – **majority voting** is employed for classifying the leaf.
- There are no samples left.



Decision Trees – Problem Type 3

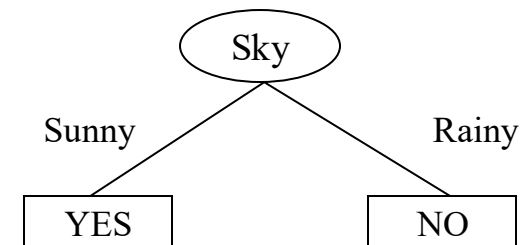
Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes
Rainy	Warm	Normal	Breeze	Same	????

Predict the class for new input instance

Follow the tree:

- Check **Sky** → it's **Rainy** → go to the right leaf → **NO**.

Predicted class: *EnjoySport?* = NO.



Problem Type 4 – Evaluate the model

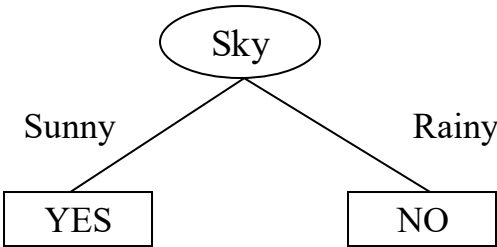
Confusion Matrix

- The evaluation discussed under module 4 is applicable here
- By default , model’s predictive quality is evaluated **only by the unseen validation/test instances.**
- When training performance (eg., accuracy) is required for comparison, the training instances are used for assessment

Actual class\Predicted class	C_1	$\neg C_1$
C_1	True Positives (TP)	False Negatives (FN)
$\neg C_1$	False Positives (FP)	True Negatives (TN)

• $CM_{i,j}$ indicates the number of tuples of class i that were labelled by the classifier as class j .

Sky	AirTemp	Humidity	Wind	Forecast	EnjoySport?
Sunny	Warm	Normal	Strong	Same	Yes
Sunny	Warm	High	Strong	Same	Yes
Rainy	Cold	High	Strong	Change	No
Sunny	Warm	High	Strong	Change	Yes
Rainy	Warm	Normal	Breeze	Same	Yes



Practice Exercises (for Students)

1. For this similar problem discussed in class build a decision tree classifier using ID3 algorithm ie., Information gain and entropy measures. Grow the complete decision tree.
2. Can "Object ID" be used as one of key feature to construct the tree? Justify your answer on the results of entropy calculated.
3. Use the test data to construct the confusion matrix and find the precision , recall, F-score.
4. Compare the training accuracy vs test accuracy for the model built in part 1)

	Training Data			
Object ID	Shape	Colour	Size	Action
08101	Round	Green	Small	Reject
08102	Square	Black	Big	Allow
08103	Square	Brown	Big	Allow
08104	Round	Brown	Small	Reject
08105	Square	Green	Big	Allow
08105	Square	Brown	Small	Reject
08106	Oval	Green	Big	Reject
08107	Oval	Brown	Small	Allow
08108	Oval	Green	Small	Reject

	Test Data / Prune Set/ Validation Set			
Object ID	Shape	Colour	Size	Action
08109	Oval	Black	Small	Reject
08110	Round	Brown	Big	Allow
08111	Square	Brown	Big	Allow
08112	Oval	Green	Small	Allow

Answer - Hint

Final ID3 tree

- Shape
 - Round → Reject
 - Square → Size
 - Big → Allow
 - Small → Reject
 - Oval → Colour
 - Brown → Allow
 - Green → Reject

No, **Object ID cannot be used as a splitting feature** because it is just an identifier. Each ID is unique, so splitting on it will simply memorize the training data (entropy = 0) without learning any real pattern. This causes **overfitting** and the model won't generalize to new data.



Confusion matrix (Positive = Allow):

	Pred Allow	Pred Reject
True Allow	1 (TP)	2 (FN)
True Reject	0 (FP)	1 (TN)

- **Training accuracy:** 100% (pure leaves after two levels).
- **Test accuracy:** 50%.

Interpretation: the tree fits the small training set perfectly and **overfits**—it doesn't generalize as well to new combinations (e.g., unseen "Oval & Black"). This also illustrates why **using Object ID would make overfitting worse** (near-100% training accuracy with poor/undefined test behavior for new IDs).

Alternative to Entropy: Gini Index

- The **Gini Index** (used in decision trees like CART) measures the impurity of a dataset.

Formally, for a node t :

$$Gini(t) = 1 - \sum_{i=1}^c (p(i|t))^2$$

where:

- c = number of classes
- $p(i|t)$ = proportion of samples in node t that belong to class i

Node N_1	Count
Class=0	0
Class=1	6

$$Gini = 1 - (0/6)^2 - (6/6)^2 = 0$$

Node N_2	Count
Class=0	1
Class=1	5

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

Node N_3	Count
Class=0	3
Class=1	3

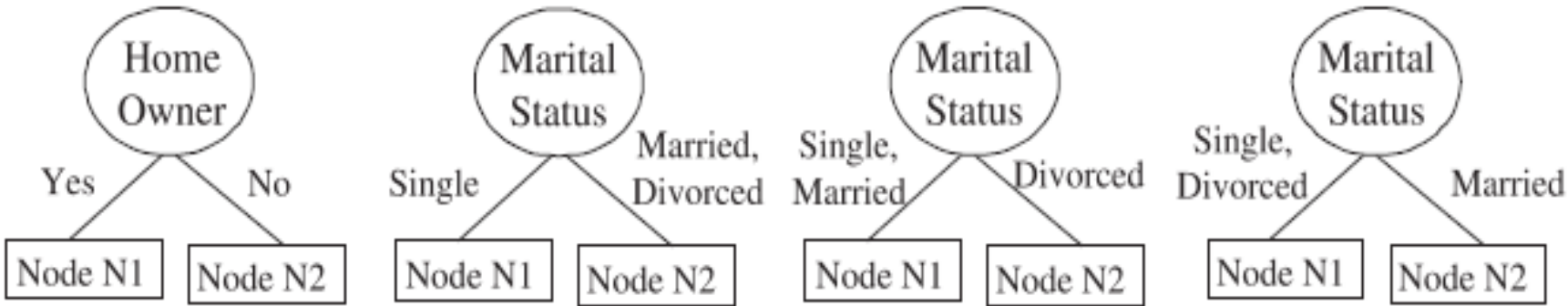
$$Gini = 1 - (3/6)^2 - (3/6)^2 = 0.5$$

	Parent
No	7
Yes	3
Gini = 0.420	

$$1 - \left(\frac{3}{10}\right)^2 - \left(\frac{7}{10}\right)^2 = 0.420.$$

Splitting of Attributes - Gini Index : Binary Splits

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes



Splitting of Attributes -Gini Index : Binary Splits

Parent Node Gini (before split)

From the full dataset of 10 records:

• Yes (Defaulted) = 3, No (Defaulted) = 7

$$Gini_{parent} = 1 - \left(\left(\frac{3}{10} \right)^2 + \left(\frac{7}{10} \right)^2 \right) \\ = 1 - (0.09 + 0.49) = 1 - 0.58 = 0.420$$

Splitting Attribute: HomeOwner

Gini index for the child nodes

$$N1 = 1 - (0/3)^2 - (3/3)^2 = 0$$

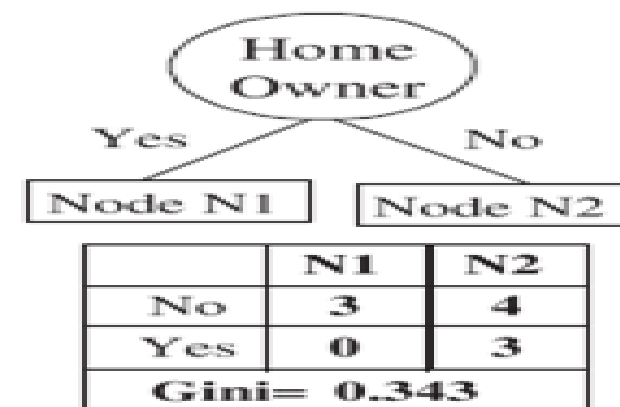
$$N2 = 1 - (3/7)^2 - (4/7)^2 = 0.490$$

Weighted Gini Index for children

$$I(\text{children}) = \sum_{j=1}^k \frac{N(v_j)}{N} I(v_j)$$

$$(3/10) \times 0 + (7/10) \times 0.490 = 0.343,$$

ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

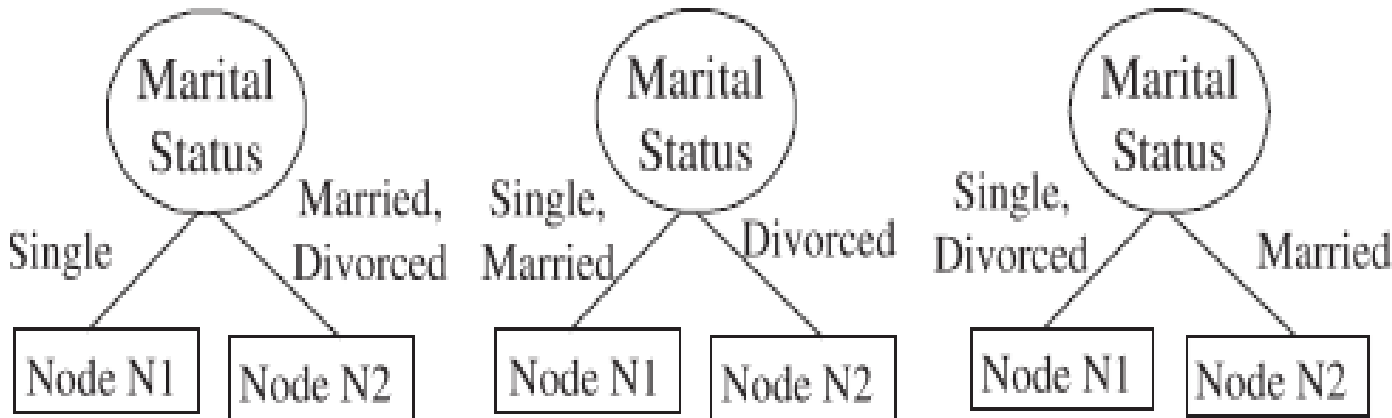


➤ The gain using Home Owner as splitting attribute is $0.420 - 0.343 = 0.077$

$$Gain = Gini_{parent} - Gini_{children}$$

Splitting of Nominal Attributes :Gini Index : Binary Splits

Splitting Attribute: MaritalStatus



ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

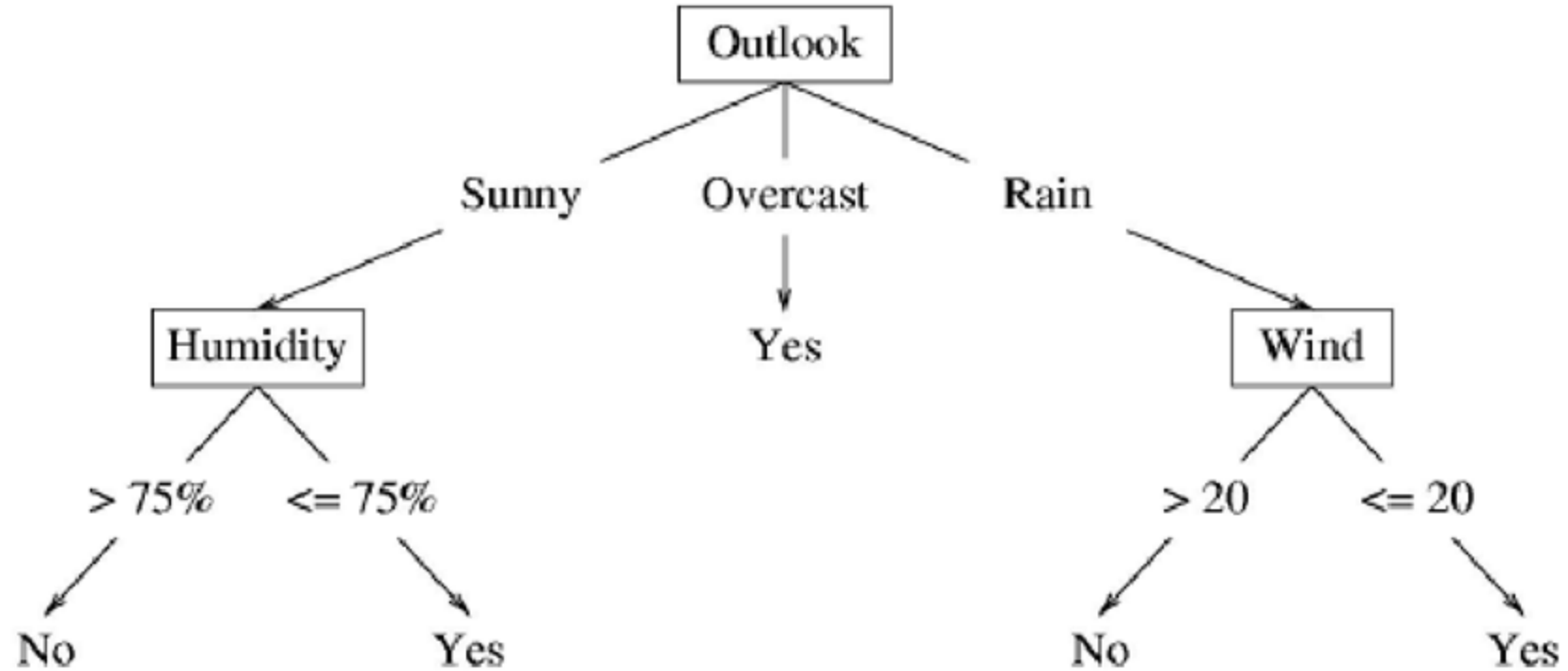
	N1	N2
No	3	4
Yes	2	1
Gini= 0.400		

	N1	N2
No	6	1
Yes	2	1
Gini= 0.400		

	N1	N2
No	4	3
Yes	3	0
Gini= 0.343		

- Based on these results, Home Owner and the last binary split using Marital Status are clearly the best candidates, since they both produce the lowest weighted average Gini index

Decision Trees – Dealing with Continuous Values



Decision Trees – Dealing with Continuous Values – Problem Type 5

- Given a continuous-valued attribute A , dynamically create a new attribute A_c

$A_c = \text{True if } A < c,$

False otherwise

- How to determine threshold value c ?

Way 1: Multi-way Split

- Example. *Temperature* in the *PlayTennis* example
 - Sort the examples according to *Temperature*

<i>Temperature</i>	40	48		60	72	80		90
<i>PlayTennis</i>	No	No	54	Yes	Yes	Yes	85	No

- Determine candidate thresholds by averaging consecutive values where there is a change in classification: $(48+60)/2=54$ and $(80+90)/2=85$

Decision Trees – Dealing with Continuous Values – Problem Type 5

- Given a continuous-valued attribute A , dynamically create a new attribute A_c

$A_c = \text{True if } A < c, \text{ False otherwise}$

- How to determine threshold value c ?

Way 2: Binary Splits

- Example. *Annual Income*

Sort the examples according to *Annual Income*

Linearly scan all possible threshold, to determine best split point where the impurity is the least



ID	Home Owner	Marital Status	Annual Income	Defaulted?
1	Yes	Single	125000	No
2	No	Married	100000	No
3	No	Single	70000	No
4	Yes	Married	120000	No
5	No	Divorced	95000	Yes
6	No	Single	60000	No
7	Yes	Divorced	220000	No
8	No	Single	85000	Yes
9	No	Married	75000	No
10	No	Single	90000	Yes

Decision Trees – Dealing with Continuous Values – Problem Type 5

Way 2: Binary Splits

- Example. *Annual Income*
- Candidate split positions are identified by taking the midpoints between two adjacent sorted values: 55, 65, 72, and so on.
- Extra split **before the smallest value** (and sometimes after the largest value).

Class		No		No		No		Yes		Yes		Yes		No		No		No		No			
		Annual Income (in '000s)																					
Sorted Values	→	60		70		75		85		90		95		100		120		125		220			
		55		65		72.5		80		87.5		92.5		97.5		110		122.5		172.5		230	
Split Positions	→																						
		<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
	Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
	No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0

Decision Trees – Dealing with Continuous Values – Problem Type 5

Way 2: Binary Splits

Split at 55

- **Left node (≤ 55):** no data points $\rightarrow$ (Yes = 0, No = 0).
 - By definition, $Gini(\text{Empty}) = 0$.
- **Right node (> 55):** all 10 samples $\rightarrow$ (Yes = 3, No = 7).
 - $p(\text{Yes}) = 3/10 = 0.3$, $p(\text{No}) = 7/10 = 0.7$.
 - $Gini(R) = 1 - (0.3^2 + 0.7^2)$

$$= 1 - (0.09 + 0.49) = 1 - 0.58 = 0.42$$

Weighted Gini for the split

$$Gini_{split} = \frac{0}{10} \cdot 0 + \frac{10}{10} \cdot 0.42 = 0.42$$

Split 97.5 → partitions:

- Left: $\leq 97.5 \rightarrow$ incomes {60,70,75,85,90,95} $\rightarrow$ 6 samples: Yes = 3, No = 3.
- Right: $> 97.5 \rightarrow$ incomes {100,120,125,220} $\rightarrow$ 4 samples: Yes = 0, No = 4.

Compute Gini for each child:

Left node L :

$$p_{Yes|L} = 3/6 = 0.5, p_{No|L} = 3/6 = 0.5$$

$$Gini(L) = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5$$

Right node R :

$$p_{Yes|R} = 0/4 = 0, \quad p_{No|R} = 1$$

$$Gini(R) = 1 - (0^2 + 1^2) = 0$$

Weighted Gini of the split:

$$Gini_{split} = \frac{N_L}{N} Gini(L) + \frac{N_R}{N} Gini(R) = \frac{6}{10} \cdot 0.5 + \frac{4}{10} \cdot 0 = 0.6 \cdot 0.5 = 0.30$$

Sorted Values Split Positions	Class	No		No		No		Yes		Yes		Yes		No		No		No		No			
	Annual Income (in '000s)																						
	60		70		75		85		90		95		100		120		125		220				
	55		65		72.5		80		87.5		92.5		97.5		110		122.5		172.5		230		
	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	
	Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
	No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
	Gini	0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

Classification Error

Classification Error (impurity measure)

$$\text{Error}(t) = 1 - \max_i p_i(t)$$

- Intuition:
 - $\max_i p_i(t)$ is the fraction of samples correctly classified if you always predict the majority class.
 - The rest, $1 - \max_i p_i(t)$, are misclassified $\rightarrow$ classification error.

Node N_1	Count
Class=0	0
Class=1	6

$$\text{Error} = 1 - \max[0/6, 6/6] = 0$$

Node N_2	Count
Class=0	1
Class=1	5

$$\text{Error} = 1 - \max[1/6, 5/6] = 0.167$$

Node N_3	Count
Class=0	3
Class=1	3

$$\text{Error} = 1 - \max[3/6, 3/6] = 0.5$$

Note: In practice, **Gini Index** or **Entropy** are often preferred over simple classification error when growing a decision tree, because classification error is **less sensitive to changes in class proportions**.

An alternative measure: gain ratio

- SplitInformation measures the entropy of S with respect to the values of A .
- The more uniformly dispersed the data the higher it is.

Let S be the set of examples and attribute A partition S into subsets $S_1, S_2, \dots, S_c$ (one subset per value of A). Let $|S| = n$ and $|S_i|$ be the size of subset S_i .

$$\text{SplitInformation}(S, A) = - \sum_{i=1}^c \frac{|S_i|}{|S|} \log_2 \left(\frac{|S_i|}{|S|} \right)$$

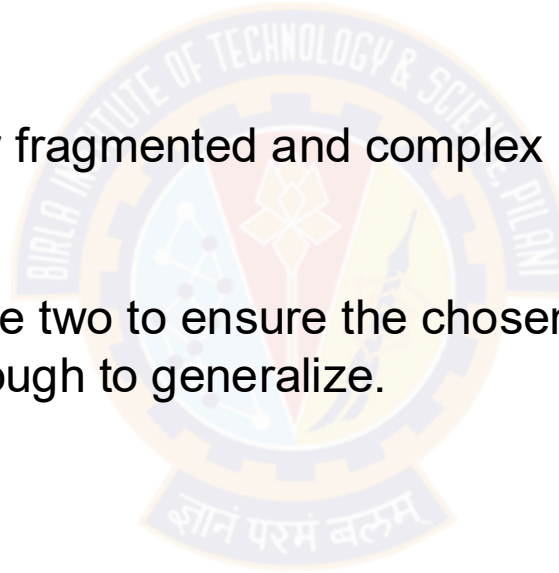
$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInformation}(S, A)}$$

(where $\text{Gain}(S, A)$ is the standard information gain: $\text{Entropy}(S) - \sum_i \frac{|S_i|}{|S|} \text{Entropy}(S_i)$).

•**Information Gain:** "How much does this attribute help me predict the target?"

•**Split Information:** "How fragmented and complex is the split created by this attribute?"

•**Gain Ratio:** Balances the two to ensure the chosen attribute is both predictive and simple enough to generalize.



Limitation of Information Gain: Ways to tackle

GainRatio penalizes attributes that split examples in many small classes such as *CustomerID*, *Date*.

Let $|S| = n$, *Customer ID* splits examples in n classes.

$$\text{SplitInformation}(S, \text{customerID}) = -[(1/n \log_2 1/n) + \dots + (1/n \log_2 1/n)] = -\log_2 1/n = \log_2 n$$

Compare with *A*, which splits data in two even classes:

$$\begin{aligned} \text{SplitInformation}(S, A) &= -[(1/2 \log_2 1/2) + (1/2 \log_2 1/2)] \\ &= -[-1/2 - 1/2] = 1 \end{aligned}$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Select the best attribute test condition among the following three attributes: **Gender**, **Car Type**, and **Customer ID**

Limitation of Information Gain: Ways to tackle: Problem Type 6

$$\text{Entropy}(\text{parent}) = -\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20} = 1.$$

If Gender is used as attribute test condition:

$$\text{Entropy}(\text{children}) = \frac{10}{20} \left[-\frac{6}{10} \log_2 \frac{6}{10} - \frac{4}{10} \log_2 \frac{4}{10} \right] \times 2 = 0.971$$

$$\text{Gain Ratio} = \frac{1 - 0.971}{-\frac{10}{20} \log_2 \frac{10}{20} - \frac{10}{20} \log_2 \frac{10}{20}} = \frac{0.029}{1} = 0.029$$

If Car Type is used as attribute test condition:

$$\begin{aligned} \text{Entropy}(\text{children}) &= \frac{4}{20} \left[-\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} \right] + \frac{8}{20} \times 0 \\ &+ \frac{8}{20} \left[-\frac{1}{8} \log_2 \frac{1}{8} - \frac{7}{8} \log_2 \frac{7}{8} \right] = 0.380 \end{aligned}$$

$$\text{Gain Ratio} = \frac{1 - 0.380}{-\frac{4}{20} \log_2 \frac{4}{20} - \frac{8}{20} \log_2 \frac{8}{20} - \frac{8}{20} \log_2 \frac{8}{20}} = \frac{0.620}{1.52} = 0.41$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Select the best attribute test condition among the following three attributes:
Gender, Car Type, and Customer ID

Limitation of Information Gain: Ways to tackle: Problem Type 6

Finally, if Customer ID is used as attribute test condition:

$$\text{Entropy(children)} = \frac{1}{20} \left[-\frac{1}{1} \log_2 \frac{1}{1} - \frac{0}{1} \log_2 \frac{0}{1} \right] \times 20 = 0$$

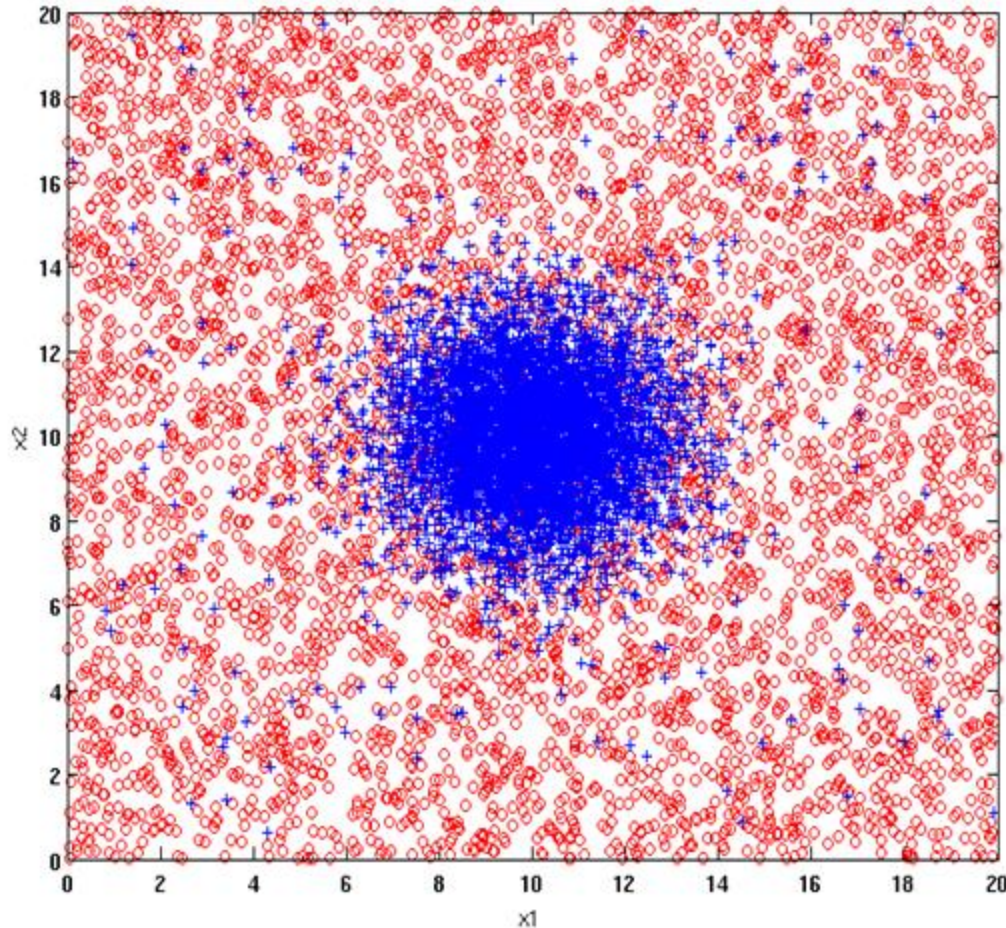
$$\text{Gain Ratio} = \frac{1 - 0}{-\frac{1}{20} \log_2 \frac{1}{20} \times 20} = \frac{1}{4.32} = 0.23$$

Customer ID	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1

Select the best attribute test condition among the following three attributes: **Gender, Car Type, and Customer ID**

Inference : Thus, even though Customer ID has the highest information gain, its gain ratio is lower than Car Type since it produces a larger number of splits

Example Data Set



Two class problem(class o and class +):

+ : 5200 instances

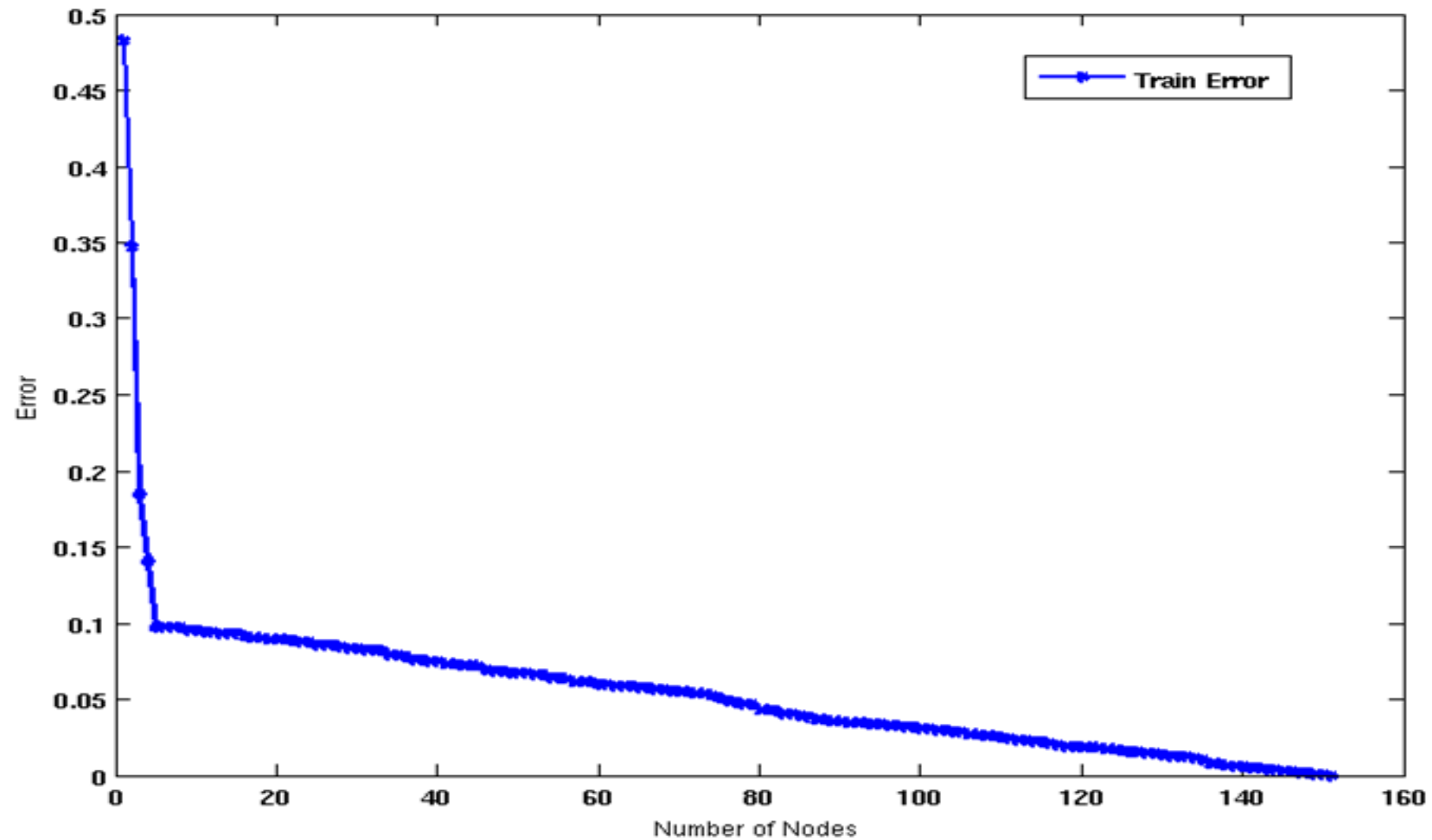
- 5000 instances generated from a Gaussian centered at (10,10)
- 200 noisy instances added

o : 5200 instances

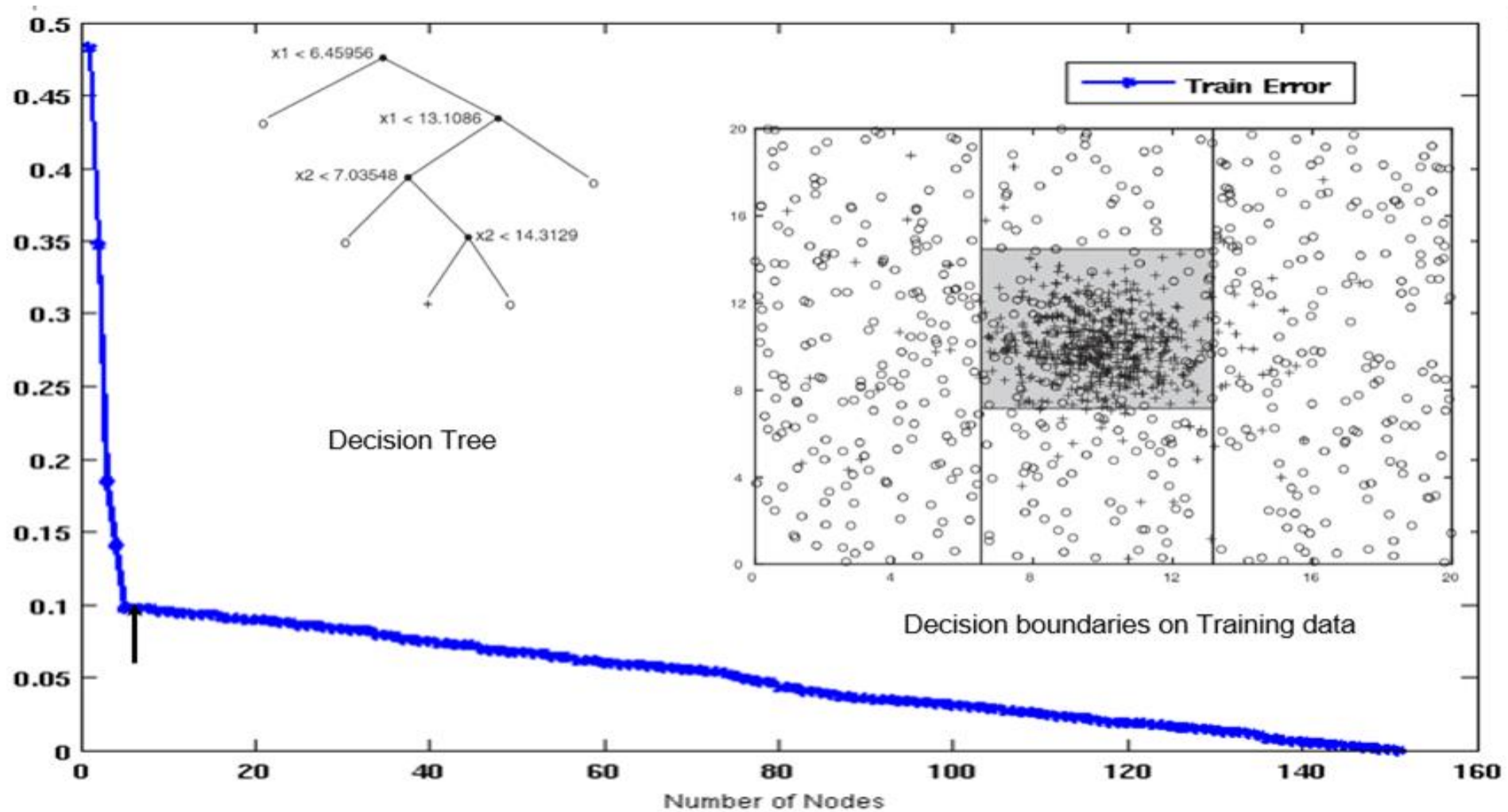
- **Generated from a uniform distribution**

10 % of the data used for training and 90% of the data used for testing

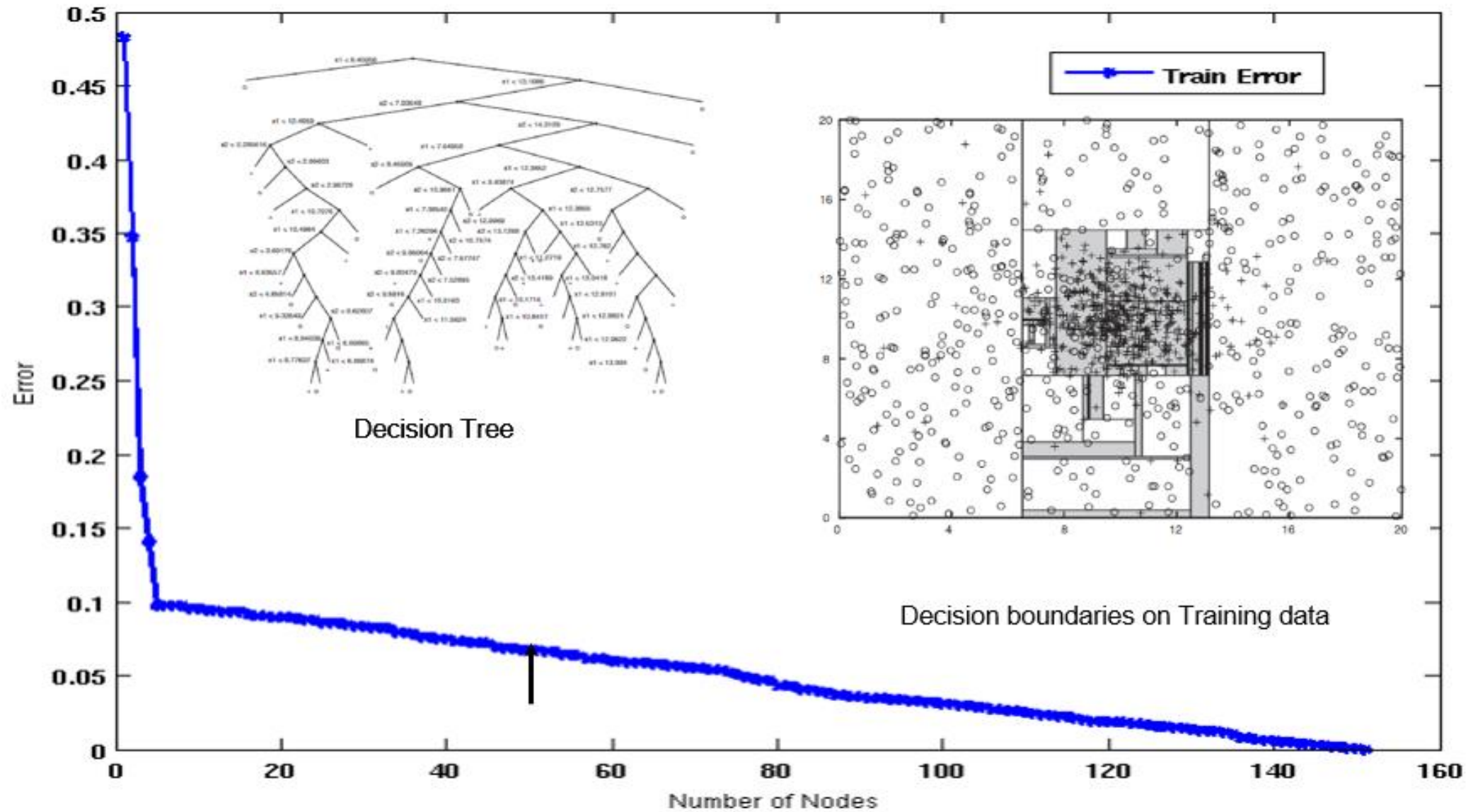
Increasing number of nodes in Decision Trees



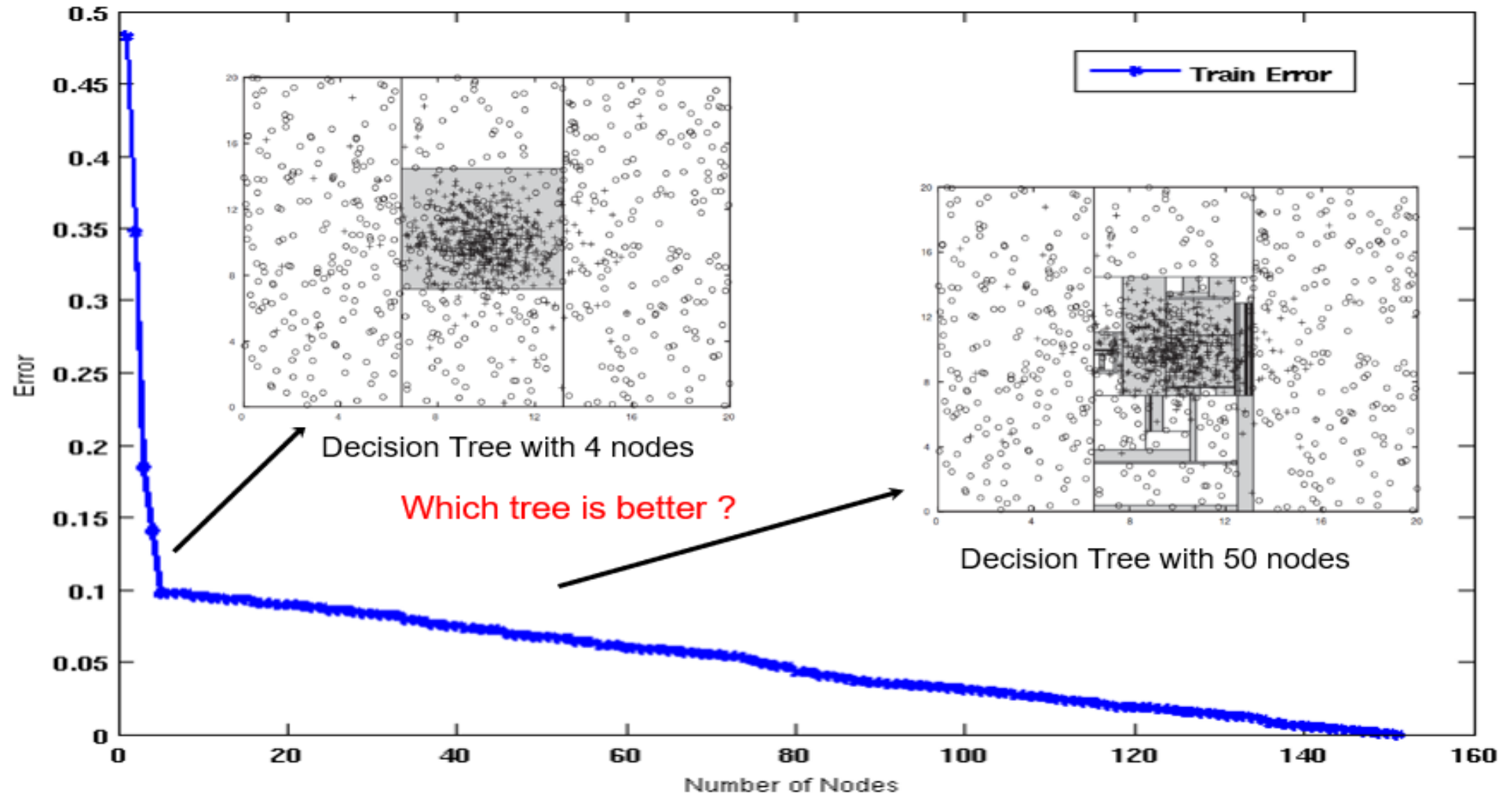
Decision Tree with 4 nodes



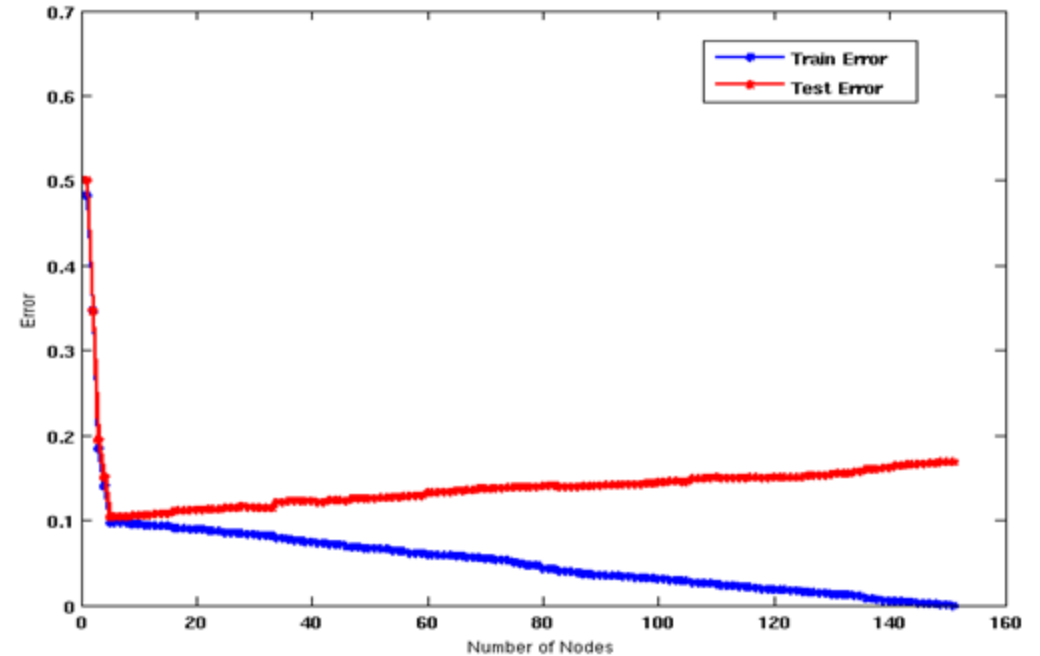
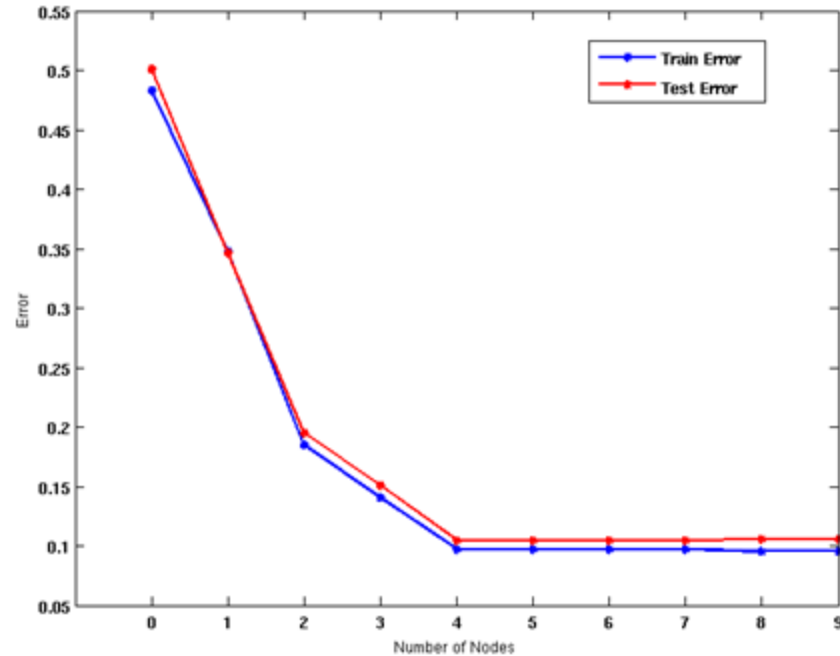
Decision Tree with 50 nodes



Which tree is better? Problem Type 7 – Model Comparison

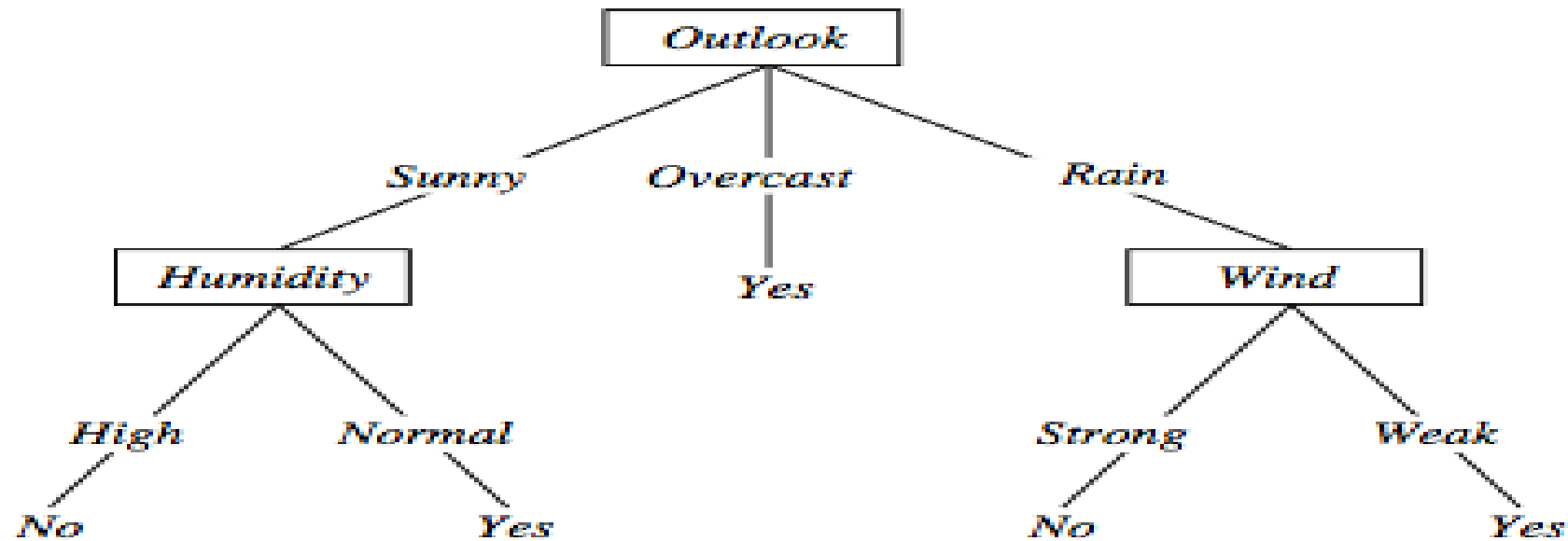


Model Overfitting



- Underfitting: when model is too simple, both training and test errors are large
- Overfitting: when model is too complex, training error is small but test error is large

Overfitting in decision trees



$\langle \text{Outlook}=\text{Sunny}, \text{Humidity}=\text{Normal}, \text{Wind}=\text{Strong}, \text{PlayTennis}=\text{No} \rangle$

For "Sunny, Humidity = Normal", the current tree says **Yes**.

But this new training example says **No** → a **contradiction**.

New noisy example causes splitting of second leaf node.

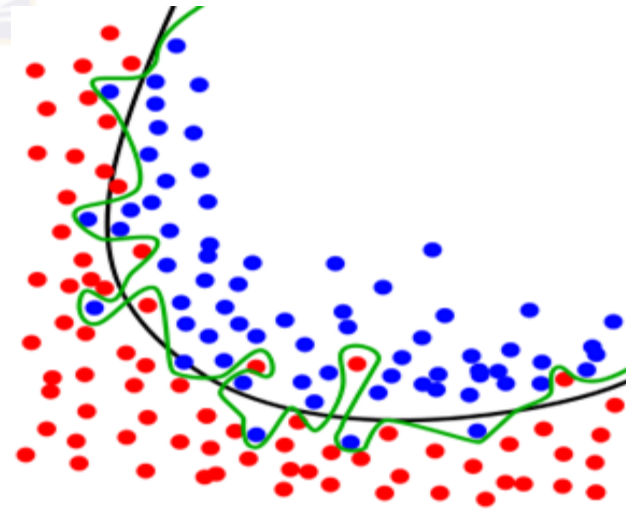
- The tree looks “perfect” on training data (because it fits the noise).
- But on **new unseen data**, accuracy may drop because the tree has become too specific.

Overfitting and Tree Pruning

- Overfitting: An induced tree may overfit the training data
 - Too many branches, may reflect anomalies due to noise or outliers in the training data
 - Poor accuracy for unseen samples

Approaches to tackle:

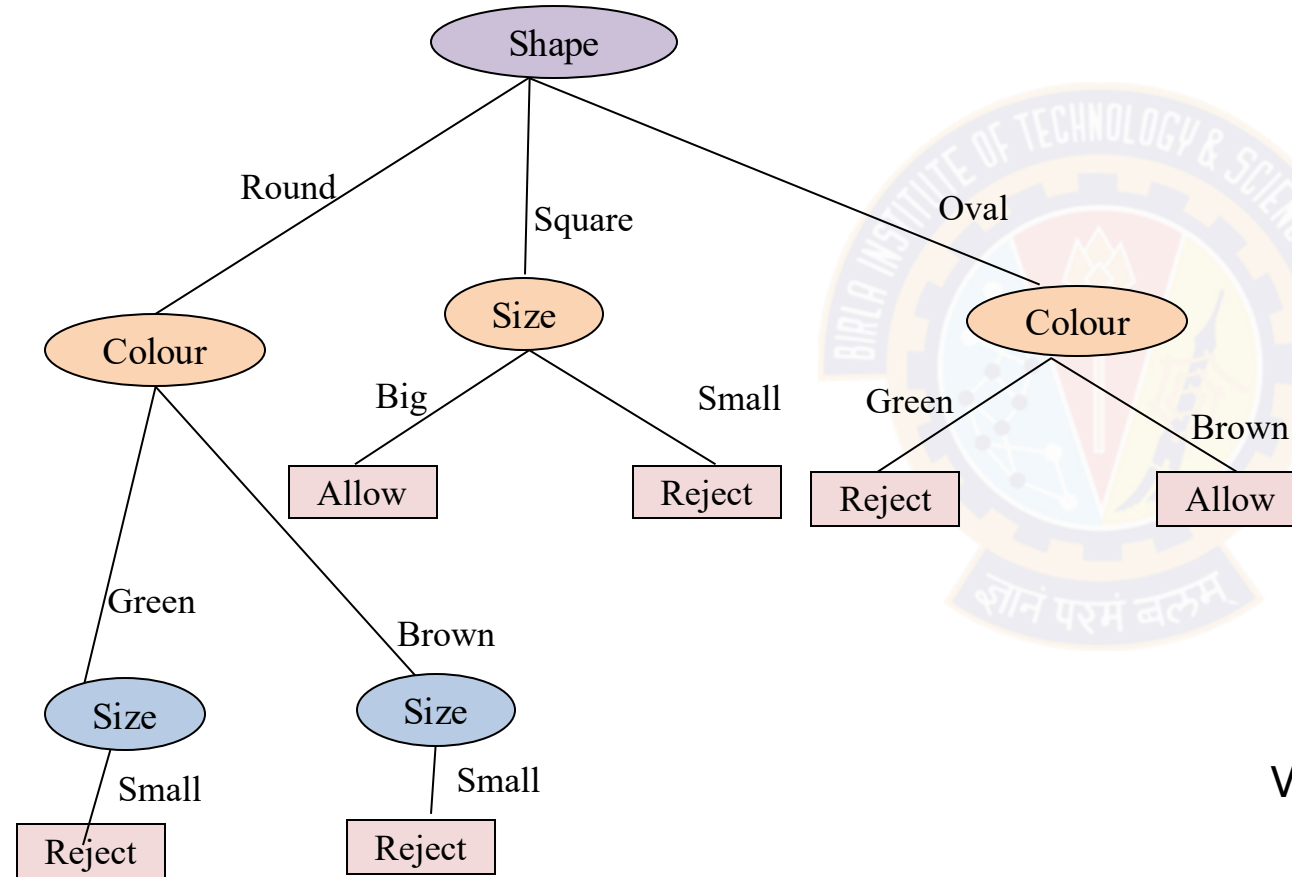
- Pruning
- Constraint on the depth of tree
- Constraint on the minimum nodes allowed at leaf



Over fitting

Assumed over fit model

• ID3



Training Set

Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Validation/Test Set

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Pre-pruning

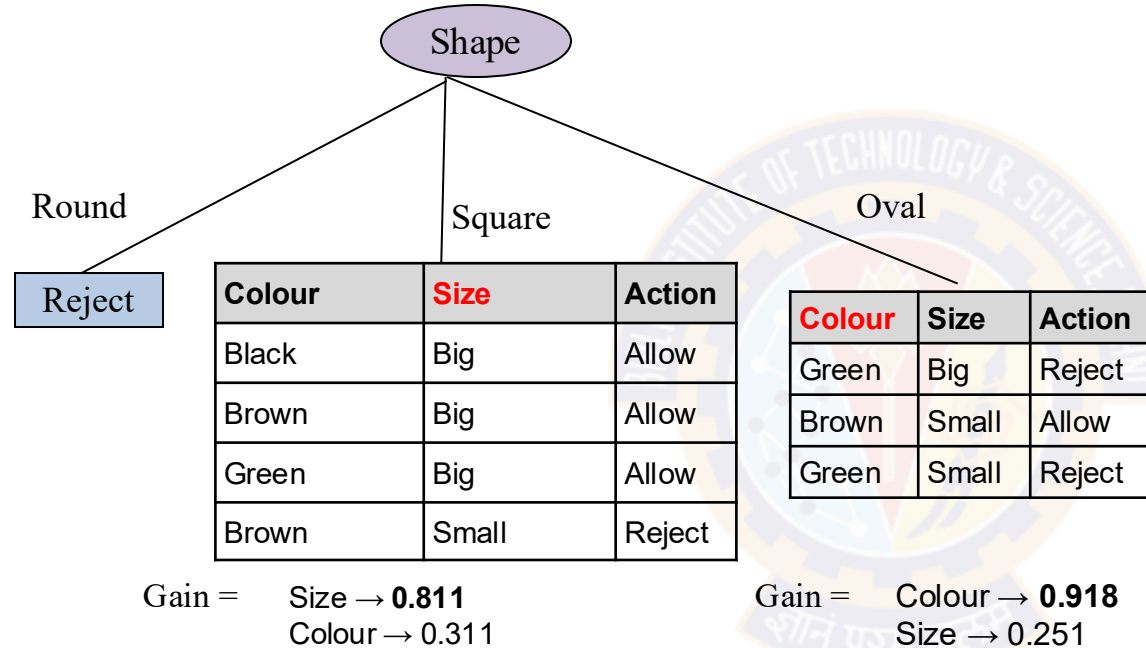
- **Tree** growth is stopped early, before it fully classifies the training data perfectly.
- Instead of letting the tree grow until every training example is separated, the algorithm uses stopping conditions to decide when to stop splitting.
- ◆ Typical Stopping Conditions
 - Stop if the **information gain (or Gini gain)** is below a threshold.
 - Stop if the node has **fewer than a minimum number of samples**.
 - Stop if tree **depth exceeds a maximum level**.
 - Stop if improvement in **accuracy/generalization** is not significant.

Choosing the **right threshold** is tricky:

- Too strict → tree becomes too shallow → **underfitting**.
- Too loose → tree still grows too much → **overfitting**.

Pre-Pruning – Problem Type 8

- *Threshold: $Gain \leq 0.85$*



Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

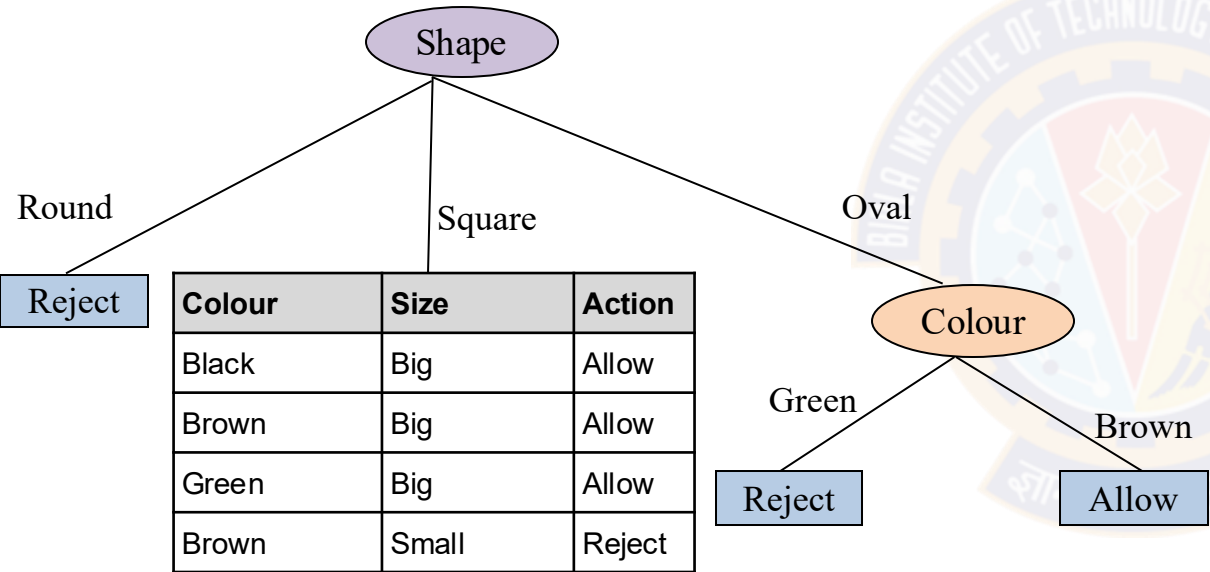
Square: $0.811 < 0.85 \rightarrow$ **do not split** the square node (it becomes a leaf).

Oval: $0.918 \geq 0.85 \rightarrow$ **split on Colour**.

Idea : While construction, Prune the nodes whose gain is less than or equal to the threshold.

Pre-Pruning – Problem Type 8

Threshold: Gain ≤ 0.85

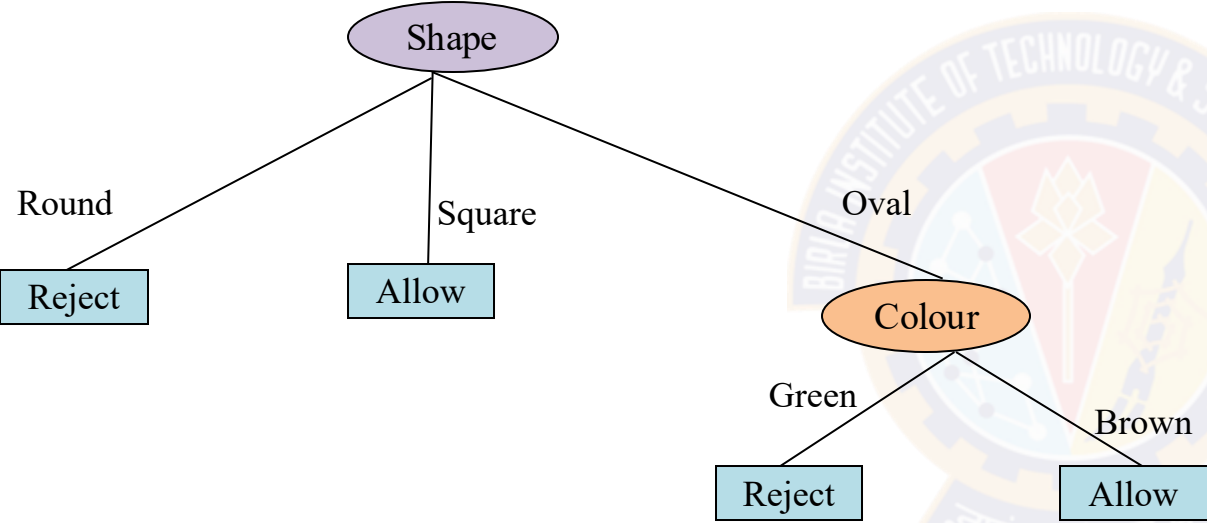


Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Pre-Pruning – Problem Type 8

Threshold: $\text{Gain} \leq 0.85$



Apply majority voting for converting the pruned subset of data into a class

Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Shape	Colour	Size	Action
Oval	Black	Small	Reject
Round	Brown	Big	Allow
Square	Brown	Big	Allow
Oval	Green	Small	Allow

Post-Pruning

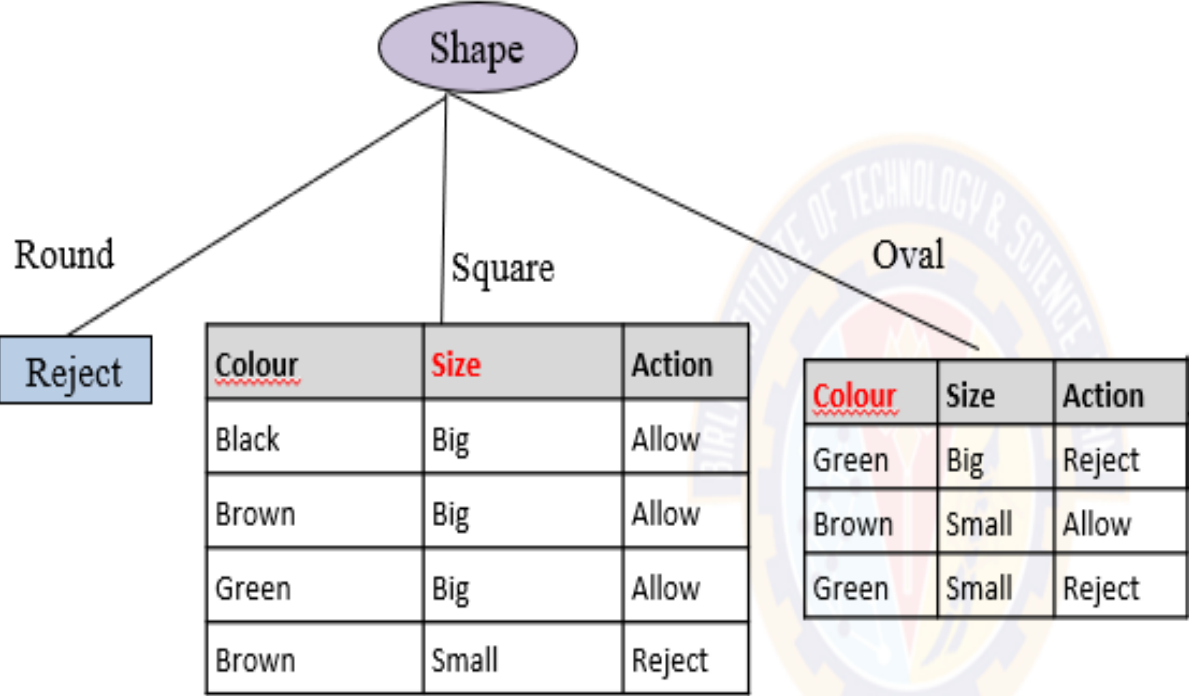
Idea :

1. Grow the tree fully: Don't stop early — keep splitting until every node is pure (or minimal samples per leaf).
2. Scan the tree bottom-up
3. At every decision node
 - Retain the attribute node & evaluate it against the prune set / validation set(used to estimate the cost)
 - Remove the attribute node & reevaluate it with the same prune set
 - If there is a reduction in error, prune the node else retain the node
4. Repeat this in other branches of the tree

Error rate is the percentage of tuples misclassified

Post-Pruning – Problem Type 9

Assumed model from the output of one intermediate iteration of the decision tree building

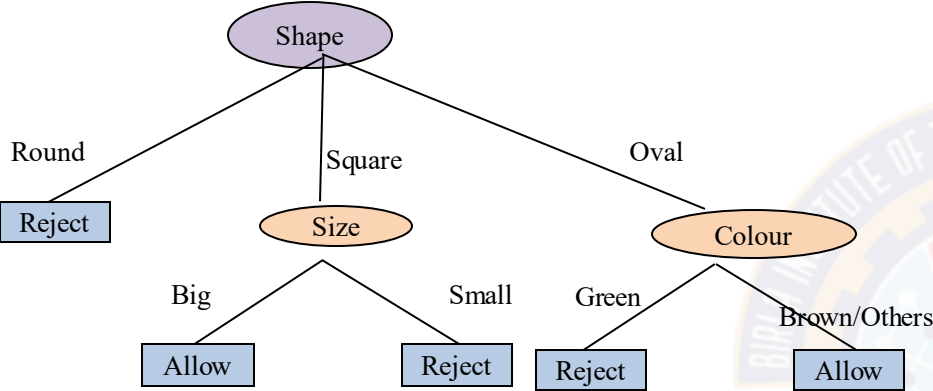


Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Prune Set

Shape	Colour	Size	Action
Oval	Black	Small	Allow
Round	Brown	Big	Reject
Square	Brown	Big	Allow
Oval	Green	Small	Reject

Post-Pruning – Problem Type 9



Shape	Colour	Size	Action
Round	Green	Small	Reject
Square	Black	Big	Allow
Square	Brown	Big	Allow
Round	Brown	Small	Reject
Square	Green	Big	Allow
Square	Brown	Small	Reject
Oval	Green	Big	Reject
Oval	Brown	Small	Allow
Oval	Green	Small	Reject

Prune Size & Predict	Prune Colour & Predict	Above Tree's Prediction	Shape	Colour	Size	Action
Allow	Reject	Allow	Oval	Black	Small	Allow
Reject	Reject	Reject	Round	Brown	Big	Reject
Allow	Allow	Allow	Square	Brown	Big	Allow
Reject	Reject	Reject	Oval	Green	Small	Reject

Minimum Description Length Principle

The best decision tree is the one that minimizes the total description length of:

- The tree itself (its structure: number of nodes, attributes used, thresholds)
- The data when encoded using the tree (the errors or exceptions that the tree cannot perfectly explain).

$$\begin{aligned} h_{MAP} &= \arg \max_{h \in H} P(D|h)P(h) \\ &= \arg \max_{h \in H} \log_2 P(D|h) + \log_2 P(h) \\ &= \arg \min_{h \in H} -\log_2 P(D|h) - \log_2 P(h) \quad (1) \end{aligned}$$

- Interesting fact from information theory:

The optimal (shortest expected coding length) code for an event with probability p is $-\log_2 p$ bits.

So interpret (1):

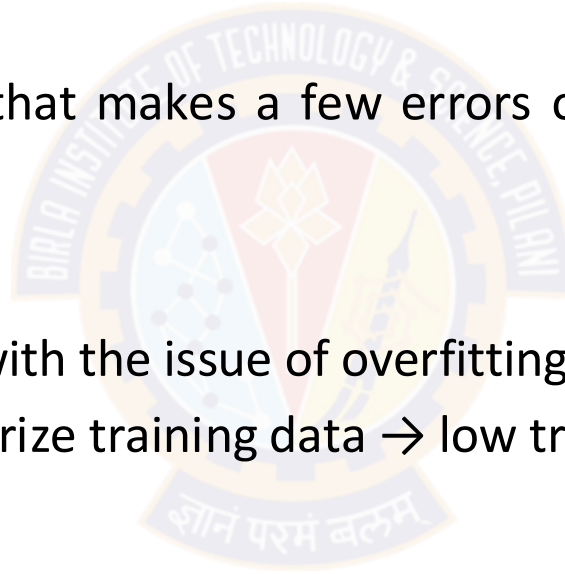
- $L_{C1}(h) = \text{length}(h) = -\log_2 P(h)$
- $L_{C2}(D|h) = \text{length}(\text{misclassifications}) = -\log_2 P(D|h)$

→ prefer the hypothesis that minimizes

$\text{length}(h) + \text{length}(\text{misclassifications})$

Minimum Description Length Principle

- MDL principle provides a way of trading off hypothesis complexity for the number of errors committed by the hypothesis.
- May select a shorter hypothesis that makes a few errors over a longer hypothesis that perfectly classifies the training data.
- Provides one method for dealing with the issue of overfitting the data
 - Large complex trees can memorize training data → low training error but poor generalization.



Minimum Description Length Principle

Occam's razor: prefer the shortest hypothesis (In decision trees, shorter trees are better, provided they still classify the data reasonably well.)

MDL: prefer the hypothesis h that minimizes

$$h_{MDL} = \operatorname{argmin}_{h \in H} L_{C_1}(h) + L_{C_2}(D|h)$$

where $L_C(x)$ is the description length of x under encoding C

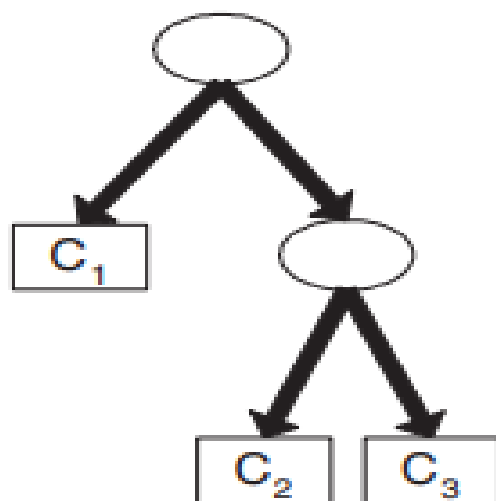
Example: H = decision trees hypothesis, D = training data labels

- $L_{C_1}(h)$ is # bits to describe tree h
- $L_{C_2}(D|h)$ is # bits to describe D given h
 - Note $L_{C_2}(D|h) = 0$ if examples classified perfectly by h . Need only describe exceptions
- Hence h_{MDL} trades off tree size for training errors

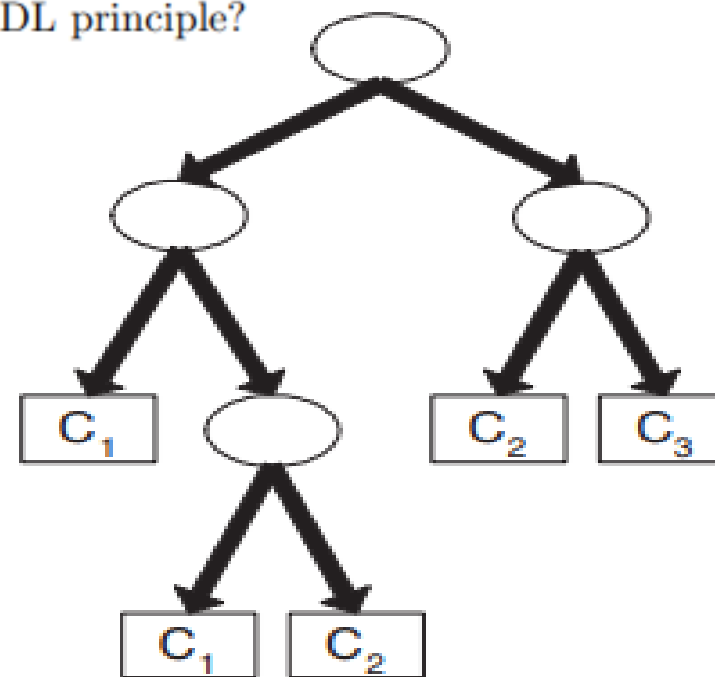
Minimum Description Length Principle Problem Type 10

9. Consider the decision trees shown in Figure 4.3. Assume they are generated from a data set that contains 16 binary attributes and 3 classes, C_1 , C_2 , and C_3 . Compute the total description length of each decision tree according to the minimum description length principle.

Which decision tree is better, according to the MDL principle?



(a) Decision tree with 7 errors



(b) Decision tree with 4 errors

Minimum Description Length Principle Problem Type 10

The **Minimum Description Length (MDL)** principle says:

The best model is the one that minimizes the total description length of the model **plus** the data given the model.

For a decision tree:

$$L(T) = L(\text{Tree Structure}) + L(\text{Errors})$$

- **Tree structure cost** → how many bits are needed to describe the tree itself (internal nodes + leaf labels).
- **Error cost** → how many bits are needed to encode the training errors (the examples that the tree misclassifies).

The tree that gives the **shorter description length** is preferred, even if it makes more errors, because the model must balance **simplicity** vs. **accuracy**.

Minimum Description Length Principle Problem Type 10

Answer:

Because there are 16 attributes, the cost for each internal node in the decision tree is:

$$\log_2(m) = \log_2(16) = 4$$

Furthermore, because there are 3 classes, the cost for each leaf node is:

$$\lceil \log_2(k) \rceil = \lceil \log_2(3) \rceil = 2$$

The cost for each misclassification error is $\log_2(n)$.

The overall cost for the decision tree (a) is $2 \times 4 + 3 \times 2 + 7 \times \log_2 n = 14 + 7 \log_2 n$ and the overall cost for the decision tree (b) is $4 \times 4 + 5 \times 2 + 4 \times 5 = 26 + 4 \log_2 n$. According to the MDL principle, tree (a) is better than (b) if $n < 16$ and is worse than (b) if $n > 16$.

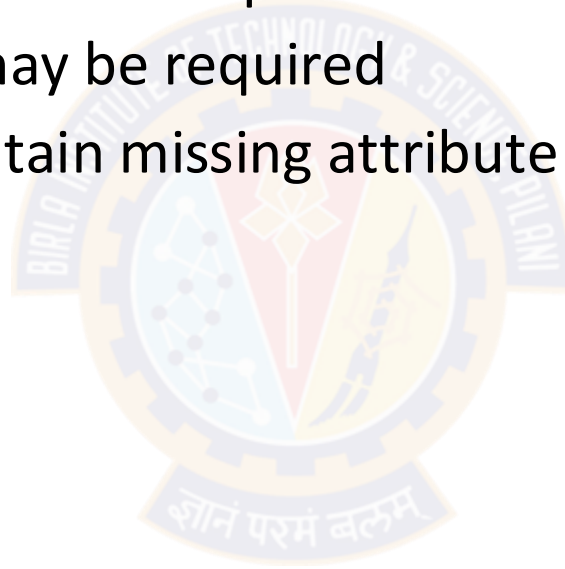
Applications

Suited for following classification problems:

- Applications whose Instances are represented by attribute-value pairs.
- The target function has discrete output values
- Disjunctive descriptions may be required
- The training data may contain missing attribute values

Real world applications

- Biomedical applications
- Manufacturing
- Banking sector
- Make-Buy decisions



Practice Exercises (for Students)

- Which of the below discretization best suits to transform the attribute "Annual Income" for decision tree construction? Use entropy as decision criteria.
 - Annual Income ($\leq 85k$, $>85k$ & $\leq 200k$, $>200k$)
 - Annual Income ($\leq 90k$, $>90k$)
- If binary split is recommended for attribute "Marital Status" which of the combination of splits best fits? Justify your comment using entropy.
- Use the results of part 1) & part 2) and Build a decision tree classifier using ID3 algorithm i.e., Information gain and entropy measures. Grow the complete decision tree.
- Use the test data to post prune the built tree. Try to prune atleast two internal nodes and choose the best of the trials

	binary	categorical	continuous	class
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Test Data / Prune Set/ Validation Set			
Owner	Marital	Income	Default
No	Single	85k	No
Yes	Single	100k	Yes
Yes	Divorced	70k	No
No	Married	90k	Yes

References

References

T1 - Chapter # 3 (Tom M. Mitchell, Machine Learning)

R1 – Chapter # 3 (PANG-NING TAN, MICHAEL STEINBACH, VIPIN KUMAR,
Introduction To Data Mining, Pearson, 2nd Edition.)

Additional References

Decision Tree

- https://www.youtube.com/watch?v=eKD5gxPPeY0&list=PLBv09BD7ez_4temBw7vLA19p3tdQH6FYO&index=1

Overfitting

- https://www.youtube.com/watch?time_continue=1&v=t56Nid85Thg
- <https://www.youtube.com/watch?v=y6SpA2Wuyt8>

Random Forest

- <https://www.stat.berkeley.edu/~breiman/RandomForests/>



Thank you